

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Kiều Văn Tuyên

**KIỂM CHỨNG CHƯƠNG TRÌNH ĐA LUỒNG
DỰA TRÊN THỰC THI TƯỢNG TRƯNG**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ thông tin

HÀ NỘI - 2023

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Kieu Van Tuyen

**VERIFICATION OF MULTITHREADED PROGRAMS
BASED ON SYMBOLIC EXECUTION**

THE BS THESIS

Major: Information Technology

Supervisor: Dr. To Van Khanh

HANOI - 2023

Lời cam đoan

Tôi là Kiều Văn Tuyên, sinh viên lớp K64-CA-CLC2, ngành Khoa học máy tính. Tôi xin cam đoan khóa luận “Kiểm chứng chương trình đa luồng dựa trên thực thi tượng trưng” là công trình nghiên cứu của bản thân tôi dưới sự hướng dẫn của TS. Tô Văn Khánh. Nội dung khóa luận là trung thực và không sao chép từ bất kỳ nguồn nào khác.

Tôi xin cam đoan mọi thông tin trong khóa luận này là trung thực và không sao chép từ bất kỳ nguồn nào khác mà không trích dẫn. Các kết quả nêu trong khóa luận là trung thực và chưa từng được công bố trước đây. Tôi xin chịu trách nhiệm về lời cam đoan này.

Hà Nội, ngày 05 tháng 05 năm 2023

Sinh viên

Kiều Văn Tuyên

Lời cảm ơn

Trước hết, tôi xin bày tỏ lòng biết ơn sâu sắc nhất đến Giảng viên, TS. Tô Văn Khánh, người đã tận tình hướng dẫn, hướng dẫn, động viên và giúp đỡ tôi trong suốt quá trình thực hiện luận án này.

Tôi xin gửi lời cảm ơn đến các thầy cô trong Khoa Công nghệ thông tin cũng như Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội đã tạo điều kiện cho tôi được học tập trong môi trường tốt và dạy tôi những điều tốt nhất để làm. Kiến thức đặc biệt quan trọng để tôi tiếp tục học tập và làm việc trong tương lai.

Tôi đặc biệt bày tỏ lòng biết ơn đến gia đình đã luôn là chỗ dựa vững chắc giúp đỡ, hỗ trợ tôi trong mọi chặng đường.

Cuối cùng, tôi cũng xin gửi lời cảm ơn đến các bạn lớp K64-CA-CLC2, các bạn trong khóa, các anh chị em trong và ngoài trường đã cùng tôi học tập, rèn luyện và giúp đỡ lẫn nhau trong suốt 4 năm đại học.

Xin chân thành cảm ơn!

Tóm tắt

Kiểm chứng chương trình, đặc biệt là kiểm chứng chương trình đa luồng, đóng vai trò quan trọng trong việc đảm bảo tính đúng đắn và an toàn của các hệ thống phần mềm hiện đại. Những lỗi tiềm ẩn trong các chương trình đa luồng, như tranh chấp tài nguyên và khoá chết, thường rất khó phát hiện thông qua các phương pháp kiểm thử thông thường. Do đó, các phương pháp như kiểm chứng mô hình, phân tích tĩnh và thực thi tượng trưng đã được nghiên cứu và phát triển nhằm giải quyết vấn đề này. Khóa luận này đề xuất một phương pháp kiểm chứng chương trình đa luồng dựa trên thực thi tượng trưng, kết hợp với việc xây dựng các ràng buộc giải quyết xung đột giữa các biến đọc và ghi trong chương trình. Phương pháp này tận dụng các ưu điểm của thực thi tượng trưng, như khả năng phân tích sâu các tương tác phức tạp giữa các luồng, đồng thời giảm thiểu nhược điểm về bùng nổ không gian tìm kiếm. Chúng tôi đã phát triển một công cụ có tên là MVS và tiến hành thực nghiệm trên các bộ dữ liệu từ cuộc thi SV-COMP, kết quả thực nghiệm cho thấy MVS hoạt động hiệu quả trên hầu hết các bài toán kiểm chứng, với tỉ lệ thành công cao trong việc phát hiện và báo cáo các lỗi tiềm ẩn. Khóa luận này không chỉ giới thiệu một phương pháp mới cho việc kiểm chứng chương trình đa luồng mà còn mở ra nhiều hướng nghiên cứu và phát triển trong tương lai. Với công cụ MVS, chúng tôi hy vọng đóng góp vào việc nâng cao chất lượng và độ tin cậy của các hệ thống phần mềm hiện đại, đồng thời cung cấp một nền tảng cho các nghiên cứu tiếp theo trong lĩnh vực kiểm chứng chương trình đa luồng.

Từ khóa: Kiểm chứng chương trình, kiểm chứng chương trình đa luồng, thực thi tượng trưng, ràng buộc giải quyết xung đột, công cụ kiểm chứng.

Mục lục

Lời cam đoan	i
Lời cảm ơn	ii
Tóm tắt	iii
Mục lục	iv
Danh sách hình vẽ	vi
Danh sách bảng	vii
Danh sách thuật toán	viii
Danh mục các từ viết tắt	ix
Chương 1 Giới thiệu	1
Chương 2 Kiến thức cơ sở	5
2.1 Giới thiệu về kiểm chứng chương trình	5
2.2 Kiểm chứng chương trình đa luồng	6
2.2.1 Chương trình đa luồng	6
2.2.2 Vấn đề trong kiểm chứng chương trình đa luồng	8
2.3 Các nghiên cứu và công cụ về kiểm chứng chương trình đa luồng . . .	8
2.4 Bộ dữ liệu kiểm chứng SV-COMP	10
2.5 Cây cú pháp trừu tượng	12
2.6 Đồ thị luồng điều khiển	13
2.7 Bộ giải SMT	14
Chương 3 Phương pháp giải quyết xung đột trong kiểm chứng chương trình đa luồng	16
3.1 Tổng quan về phương pháp đề xuất	16

3.2 Trừu tượng hóa chương trình đa luồng dựa trên kỹ thuật thực thi tượng trưng	19
3.2.1 Pha xây dựng cây cú pháp trừu tượng	19
3.2.2 Pha xây dựng đồ thị luồng điều khiển	20
3.2.3 Pha đánh chỉ mục biến	20
3.2.4 Pha trừu tượng hoá chương trình đầu vào	28
3.2.5 Pha xây dựng công thức cho ràng buộc giải quyết xung đột đọc - ghi	34
3.2.6 Pha thực thi tượng trưng	38
Chương 4 Cài đặt công cụ và Thực nghiệm	41
4.1 Cài đặt công cụ	41
4.1.1 Tiền xử lý mã nguồn	42
4.1.2 Mô đun mã hoá biến	45
4.1.3 Mô đun trừu tượng hóa chương trình	45
4.1.4 Mô đun sinh ràng buộc giải quyết xung đột	46
4.1.5 Mô đun đọc thông tin từ người dùng	46
4.1.6 Mô đun thực thi tượng trưng	47
4.1.7 Mô đun sinh báo cáo kiểm chứng	48
4.2 Kết quả thực nghiệm	48
4.2.1 Môi trường thực nghiệm	48
4.2.2 Kết quả	50
Chương 5 Kết luận	53
Tài liệu tham khảo	55

Danh sách hình vẽ

2.1	Ví dụ chương trình đa luồng đơn giản	7
2.2	Kiến trúc của cây cú pháp trừu tượng	13
2.3	Các thành phần cơ bản trong đồ thị luồng điều khiển	13
2.4	Các cấu trúc điều khiển cơ bản trong đồ thị luồng điều khiển	14
3.1	Kiến trúc tổng quan của phương pháp kiểm chứng chương trình đa luồng dựa trên kỹ thuật thực thi tượng trưng	17
3.2	Chương trình đa luồng được mã hoá	24
3.3	Cây thực thi tượng trưng của chương trình đa luồng	25
3.4	Các biến đọc có thể được gán bởi một biến ghi ngay trước nó trong cùng một luồng	31
3.5	Các biến đọc cũng có thể được gán bởi một biến ghi trong một luồng khác bất kỳ	31
3.6	Biểu diễn ràng buộc giải quyết xung đột đọc - ghi 1	35
3.7	Biểu diễn ràng buộc giải quyết xung đột đọc - ghi 2	36
3.8	Biểu diễn ràng buộc giải quyết xung đột đọc - ghi 3	36
4.1	Kiến trúc tổng quan của công cụ	41
4.2	Ví dụ về mã nguồn trước khi tiền xử lý	43
4.3	Ví dụ về mã nguồn sau khi tiền xử lý	44
4.4	Ví dụ về tệp XML chứa thông tin về điều kiện khẳng định	47
4.5	Một tệp YAML chứa thông tin về kết quả kì vọng của bài toán <i>triangular-1</i>	49
4.6	Kết quả kiểm chứng của một bài toán	52

Danh sách bảng

4.1	Kết quả thực nghiệm trên các tập dữ liệu <i>pthread</i> , <i>pthread-nondet</i> , <i>pthread-deagle</i> , và <i>pthread-atomic</i>	50
-----	---	----

Danh sách các Thuật toán

1	Xây dựng cây cú pháp trừu tượng từ mã nguồn đầu vào	19
2	Xây dựng đồ thị luồng điều khiển từ AST	21
3	Đánh chỉ mục biến	23
4	Đồng bộ chỉ mục biến	27
5	Trừu tượng hóa chương trình đầu vào	32
6	Biểu diễn mối quan hệ đọc - ghi ζ	33
7	Xây dựng công thức giải quyết xung đột đọc - ghi	37
8	Đọc thông tin từ tệp XML	39

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa
AST	Abstract Syntax Tree	Cây cú pháp trừu tượng
CBMC	C Bounded Model Checker	Kiểm chứng mô hình giới hạn C
CDT	C/C++ Development Tooling	Công cụ phát triển C/C++
CFG	Control Flow Graph	Đồ thị luồng điều khiển
FOL	First-Order Logic	Logic bậc nhất
MVS	Multi-threaded Verification based on Symbolic Execution	Kiểm chứng đa luồng dựa trên thực thi tượng trưng
SAT	Satisfiability	Thỏa mãn
SMT	Satisfiability Modulo Theories	Lý thuyết mô đun thỏa mãn
SV-COMP	Software Verification Competition	Cuộc thi kiểm chứng phần mềm
UNSAT	Unsatisfiability	Không thỏa mãn

Chương 1

Giới thiệu

Kiểm chứng chương trình [1] là một lĩnh vực quan trọng trong ngành công nghệ phần mềm, tập trung vào việc đảm bảo tính đúng đắn và an toàn của các chương trình máy tính. Với sự phát triển nhanh chóng của công nghệ và sự phức tạp ngày càng tăng của các hệ thống phần mềm, nhu cầu về các phương pháp và công cụ kiểm chứng hiệu quả ngày càng trở nên cấp thiết [2]. Các lỗi phần mềm có thể gây ra những hậu quả nghiêm trọng, từ việc làm gián đoạn dịch vụ cho đến việc gây thiệt hại về tài chính và uy tín của các tổ chức. Do đó, việc phát triển các phương pháp kiểm chứng chương trình hiệu quả là một nhiệm vụ cấp bách.

Trong các hệ thống hiện đại, chương trình đa luồng đã trở nên phổ biến do khả năng cải thiện hiệu suất và khả năng đáp ứng của chúng [3]. Tuy nhiên, tính song song và các tương tác phức tạp giữa các luồng (threads) làm tăng nguy cơ xuất hiện các lỗi khó phát hiện như tranh chấp tài nguyên (race conditions) [4] và khoá chết (deadlocks) [5]. Những lỗi này có thể không xuất hiện trong quá trình kiểm thử thông thường mà chỉ xuất hiện trong các điều kiện cụ thể và hiếm gặp. Vì vậy, việc kiểm chứng chương trình đa luồng đòi hỏi các phương pháp tiếp cận đặc biệt để có thể phân tích và đảm bảo tính đúng đắn của chương trình. Không giống như chương trình tuần tự, chương trình đa luồng có thể có nhiều luồng thực thi cùng một lúc, tương tác với nhau thông qua việc chia sẻ dữ liệu và tài nguyên, điều này tạo ra một số thách thức trong việc kiểm chứng chương trình đa luồng. Một trong những thách thức lớn nhất là việc phải xác định tất cả các trạng thái có thể của chương trình, từ đó kiểm tra xem chương trình có thỏa mãn các tính chất an toàn và điều kiện khẳng định của người dùng hay không.

Từ lâu nay, kiểm chứng chương trình đa luồng đã trở thành một lĩnh vực nghiên cứu quan trọng trong ngành kiểm chứng phần mềm [6]. Nhiều phương pháp và công cụ đã được đề xuất để giải quyết vấn đề này. Một vài phương pháp tiếp cận bằng cách sử dụng phân tích tĩnh [7] để kiểm tra tính đúng đắn của chương trình, ưu điểm của phương pháp này là có thể kiểm tra toàn bộ không gian trạng thái của chương trình mà không cần thực thi chương trình. Tuy nhiên, phương

pháp này thường gặp khó khăn trong việc xử lý các chương trình lớn và phức tạp. Một số phương pháp khác sử dụng phương pháp phân tích động [8] để kiểm tra tính đúng đắn của chương trình, ưu điểm của phương pháp này là có thể xử lý các chương trình lớn và phức tạp hơn, tuy nhiên, phương pháp này thường không thể kiểm tra toàn bộ không gian trạng thái của chương trình. Gần đây, một số nhóm nghiên cứu đề xuất phương pháp kiểm chứng mô hình (model checking) [9] để kiểm tra tính đúng đắn của chương trình đa luồng. Phương pháp này biểu diễn chương trình dưới dạng mô hình và kiểm tra các tính chất của mô hình đó. Mặc dù phương pháp này có thể kiểm tra toàn bộ không gian trạng thái của chương trình, nhưng việc xây dựng mô hình chính xác của chương trình đa luồng là một vấn đề khó khăn. Ngoài ra, phương pháp kiểm chứng dựa trên thực thi tượng trưng (symbolic execution) [10] cũng được đề xuất để kiểm tra tính đúng đắn của chương trình đa luồng. Phương pháp này biểu diễn các trạng thái của chương trình dưới dạng công thức logic vị từ và sử dụng bộ giải SMT để kiểm tra các tính chất của chương trình. Phương pháp này có thể kiểm tra toàn bộ không gian trạng thái của chương trình và có thể xử lý các chương trình lớn và phức tạp, nhưng nhược điểm là gặp khó khăn với các chương trình lớn và phức tạp do sự bùng nổ của không gian tìm kiếm.

Song song với sự phát triển của các phương pháp kiểm chứng chương trình đa luồng, nhiều công cụ kiểm chứng chương trình đa luồng cũng đã được phát triển để hỗ trợ việc kiểm chứng chương trình đa luồng. Các công cụ này thường hỗ trợ nhiều tính năng như kiểm tra tính đúng đắn của chương trình, tìm kiếm lỗi, và sinh báo cáo kiểm chứng. Một số công cụ nổi tiếng như CBMC [11], Yogar-CBMC [12], ThreadSanitizer [13] và KLEE [14] đã được sử dụng rộng rãi trong cộng đồng kiểm chứng phần mềm để kiểm tra tính đúng đắn của chương trình đa luồng. CBMC là một công cụ kiểm chứng chương trình mã nguồn mở, hỗ trợ kiểm chứng chương trình đa luồng bằng phương pháp kiểm chứng mô hình. CBMC có thể kiểm tra các thuộc tính an toàn và bảo mật, nhưng gặp hạn chế với các chương trình phức tạp và lớn. Yogar-CBMC là một phiên bản mở rộng của CBMC, nó được thiết kế để kiểm chứng các chương trình đa luồng, giúp cải thiện khả năng phát hiện các lỗi liên quan đến đồng thời. Tuy nhiên, việc triển khai và sử dụng Yogar-CBMC vẫn còn phức tạp và yêu cầu nhiều tài nguyên. ThreadSanitizer là một công cụ kiểm chứng chương trình đa luồng của Google, giúp phát hiện các lỗi liên quan đến đồng thời như tranh chấp tài nguyên và khoá chết. KLEE là một công cụ kiểm chứng

chương trình mã nguồn mở, hỗ trợ kiểm chứng chương trình đa luồng bằng phương pháp kiểm chứng dựa trên thực thi tượng trưng. KLEE có thể kiểm tra các tính chất an toàn và bảo mật của chương trình, nhưng gặp khó khăn với các chương trình lớn và phức tạp.

Mặc dù các phương pháp và công cụ kiểm chứng chương trình đa luồng đã đạt được nhiều tiến bộ, nhưng vẫn còn nhiều thách thức cần được giải quyết. Một trong những thách thức lớn nhất là việc xử lý các chương trình lớn và phức tạp. Các chương trình lớn và phức tạp thường chứa nhiều cấu trúc điều khiển phức tạp và có thể chứa hàng trăm hoặc thậm chí hàng nghìn dòng mã nguồn. Việc kiểm tra tính đúng đắn của các chương trình lớn và phức tạp đòi hỏi các phương pháp và công cụ kiểm chứng mạnh mẽ và hiệu quả. Một số phương pháp và công cụ hiện đại đã được đề xuất để giải quyết vấn đề này, nhưng vẫn còn nhiều hạn chế cần được khắc phục. Do đó, việc đề xuất một phương pháp mới kết hợp các ưu điểm của các phương pháp hiện có và khắc phục những hạn chế của chúng là cần thiết. Trong khóa luận này, chúng tôi đề xuất một phương pháp kiểm chứng chương trình đa luồng dựa trên thực thi tượng trưng, kết hợp với việc xây dựng các ràng buộc giải quyết xung đột giữa các biến đọc và ghi trong chương trình. Phương pháp này giúp giảm không gian tìm kiếm và tăng khả năng xử lý của công cụ kiểm chứng.

Trước tiên, chúng tôi xây dựng cây cú pháp trừu tượng của chương trình đa luồng để biểu diễn cấu trúc của chương trình. Sau đó, đồ thị luồng điều khiển của chương trình sẽ được xây dựng để biểu diễn các luồng thực thi của chương trình. Tiếp theo, chúng tôi đánh chỉ mục các biến đọc và ghi trong chương trình để xác định các trạng thái của chương trình. Bước tiếp theo, chương trình C/C++ đầu vào sẽ được trừu tượng hóa chương trình dưới dạng công thức logic vị từ để biểu diễn các ràng buộc của chương trình. Cuối cùng, chúng tôi thực thi tượng trưng chương trình để kiểm tra tính đúng đắn của chương trình. Kết quả của quá trình kiểm chứng sẽ được sinh ra dưới dạng báo cáo kiểm chứng, cung cấp thông tin chi tiết về các lỗi tiềm ẩn trong chương trình. Sự khác biệt giữa phương pháp kiểm chứng của chúng tôi và các phương pháp kiểm chứng hiện có là chúng tôi kết hợp việc xây dựng ràng buộc giải quyết xung đột giữa các biến đọc và ghi trong chương trình, giúp giảm không gian tìm kiếm và tăng khả năng xử lý của công cụ kiểm chứng.

Phần còn lại của khóa luận được tổ chức như sau. Chương 2 trình bày các kiến

thức nền tảng về kiểm chứng chương trình đa luồng, bao gồm các phương pháp kiểm chứng chương trình đa luồng hiện có và các công cụ kiểm chứng chương trình đa luồng phổ biến. Chương 3 trình bày phương pháp kiểm chứng chương trình đa luồng dựa trên thực thi tượng trưng mà chúng tôi đề xuất. Chương 4 trình bày cài đặt công cụ kiểm chứng chương trình đa luồng dựa trên phương pháp mà chúng tôi đề xuất, đồng thời trình bày kết quả thực nghiệm của công cụ kiểm chứng này. Cuối cùng, chương 5 trình bày kết luận và hướng phát triển trong tương lai.

Chương 2

Kiến thức cơ sở

2.1 Giới thiệu về kiểm chứng chương trình

Kiểm chứng chương trình là một quy trình kỹ thuật nhằm đảm bảo rằng một chương trình máy tính hoạt động đúng theo các yêu cầu đã được xác định. Quá trình này không chỉ tập trung vào việc phát hiện và sửa lỗi mà còn bao gồm việc xác nhận tính đúng đắn và độ tin cậy của phần mềm. Kiểm chứng chương trình có thể được thực hiện bằng nhiều phương pháp khác nhau như kiểm chứng hình thức, phân tích tĩnh, phân tích động và kiểm chứng mô hình.

Kiểm chứng hình thức [15] là một phương pháp kiểm chứng chương trình dựa trên lý thuyết hình thức. Phương pháp này sử dụng các công cụ toán học để chứng minh tính đúng đắn của chương trình. Một trong những ưu điểm của kiểm chứng hình thức là khả năng chứng minh tính đúng đắn của chương trình mà không cần phải thực thi chương trình đó. Tuy nhiên, kiểm chứng hình thức thường gặp khó khăn khi chứng minh tính đúng đắn của chương trình lớn hoặc chương trình chứa các phần tử không xác định.

Phân tích tĩnh [7] là một phương pháp kiểm chứng chương trình dựa trên việc phân tích mã nguồn của chương trình mà không cần thực thi chương trình đó. Phương pháp này thường sử dụng các công cụ phân tích mã nguồn để phát hiện lỗi cú pháp, lỗi logic, hoặc các vấn đề khác trong chương trình. Phân tích tĩnh thường nhanh hơn so với kiểm chứng hình thức vì không cần phải chứng minh tính đúng đắn của chương trình. Tuy nhiên, phương pháp này thường không thể phát hiện được các lỗi phát sinh do dữ liệu đầu vào hoặc các lỗi xảy ra trong quá trình thực thi.

Phân tích động [8] là một phương pháp kiểm chứng chương trình dựa trên việc thực thi chương trình với các bộ dữ liệu đầu vào khác nhau để phát hiện lỗi. Phương pháp này thường sử dụng các công cụ kiểm thử tự động để tạo ra các bộ dữ liệu đầu vào và thực thi chương trình với các bộ dữ liệu đó. Phân tích động thường phát hiện được nhiều lỗi hơn so với phân tích tĩnh vì có thể phát hiện được

các lỗi phát sinh do dữ liệu đầu vào hoặc các lỗi xảy ra trong quá trình thực thi. Tuy nhiên, phương pháp này thường tốn nhiều thời gian và công sức hơn so với phân tích tĩnh.

Kiểm chứng mô hình [9] là một phương pháp kiểm chứng chương trình dựa trên việc mô hình hóa chương trình thành một mô hình toán học và kiểm chứng tính đúng đắn của mô hình đó. Phương pháp này thường sử dụng các công cụ kiểm chứng mô hình để kiểm chứng tính đúng đắn của mô hình. Một trong những ưu điểm của kiểm chứng mô hình là khả năng kiểm chứng chương trình lớn hoặc chương trình chứa các phần tử không xác định. Tuy nhiên, phương pháp này thường tốn nhiều thời gian và công sức hơn so với phân tích tĩnh và phân tích động.

Kiểm chứng dựa trên thực thi tượng trưng [10] là sự kết hợp giữa kiểm chứng mô hình và phân tích động. Phương pháp này sử dụng các công cụ kiểm chứng mô hình để mô hình hóa chương trình và sử dụng các công cụ phân tích động để thực thi tượng trưng chương trình. Kiểm chứng dựa trên thực thi tượng trưng thường phát hiện được nhiều lỗi hơn so với kiểm chứng mô hình vì có thể phát hiện được các lỗi phát sinh do dữ liệu đầu vào hoặc các lỗi xảy ra trong quá trình thực thi. Trong khoá luận này, phương pháp đề xuất sẽ sử dụng kiểm chứng dựa trên thực thi tượng trưng để kiểm chứng tính đúng đắn của chương trình đa luồng.

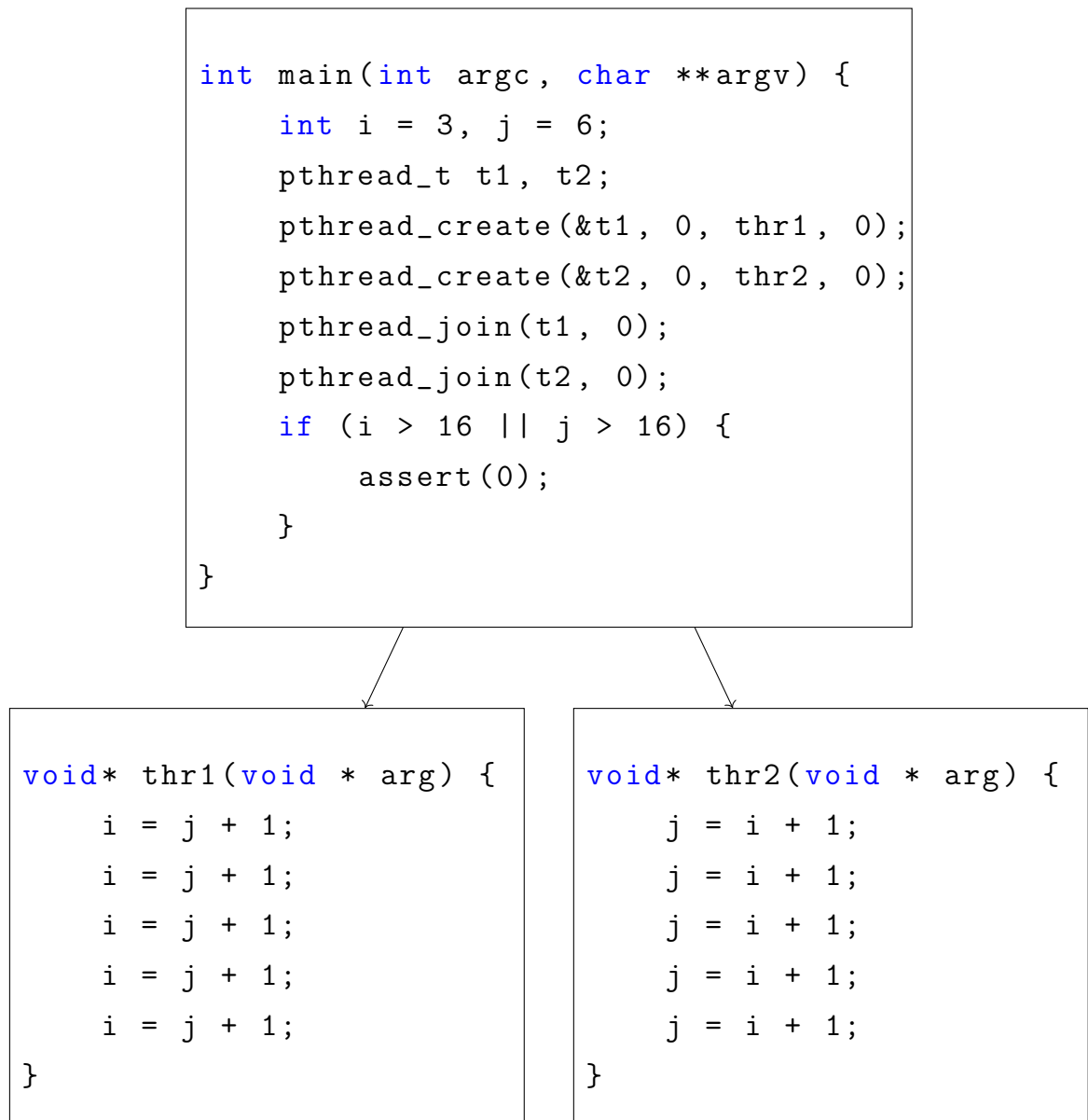
2.2 Kiểm chứng chương trình đa luồng

Kiểm chứng chương trình đa luồng là một dạng đặc biệt của kiểm chứng chương trình, nơi chương trình chứa nhiều luồng thực thi song song. Trước hết, để hiểu rõ hơn về kiểm chứng chương trình đa luồng, chúng ta cần tìm hiểu về chương trình đa luồng và các vấn đề thường gặp trong quá trình kiểm chứng chương trình đa luồng.

2.2.1 Chương trình đa luồng

Chương trình đa luồng là một chương trình máy tính mà có thể thực thi nhiều luồng đồng thời [16]. Mỗi luồng trong chương trình đa luồng thực hiện một tác vụ cụ thể và có thể tương tác với các luồng khác thông qua các biến chia sẻ hoặc thông qua cơ chế gửi nhận thông điệp. Chương trình đa luồng thường được sử dụng để tận dụng tối đa tài nguyên hệ thống và tăng cường hiệu suất của chương trình.

Xem xét một chương trình đa luồng đơn giản với hai luồng được thể hiện trên Hình 2.1, chương trình này bao gồm một hàm `main` và hai hàm con. Hàm `main` khởi tạo hai biến i và j với giá trị lần lượt là 3 và 6, sau đó tạo hai luồng mới và gọi hai hàm con `thr1` và `thr2` trong mỗi luồng. Cuối cùng, chương trình kiểm tra giá trị của i và j và kết luận rằng nếu một trong hai biến lớn hơn 16 thì chương trình sẽ báo lỗi.



Hình 2.1: Ví dụ chương trình đa luồng đơn giản

Trong chương trình trên, hai luồng `thr1` và `thr2` thực hiện các phép tính đơn giản là cộng 1 vào biến i và j mỗi lần lặp. Tuy nhiên, do việc sử dụng biến i và j mà không có cơ chế đồng bộ hóa, chương trình có thể gặp phải vấn đề đọc/ghi

không an toàn vào biến chia sẻ. Trong trường hợp này, chương trình có thể không thỏa mãn điều kiện kiểm tra cuối cùng và báo lỗi mặc dù không có lỗi xảy ra trong chương trình.

2.2.2 Vấn đề trong kiểm chứng chương trình đa luồng

Kiểm chứng chương trình đa luồng là một thách thức lớn trong lĩnh vực kiểm chứng phần mềm. Các chương trình đa luồng có khả năng thực thi song song nhiều luồng, dẫn đến sự phức tạp trong việc xác định tính đúng đắn của chương trình. Một trong những vấn đề lớn nhất là hiện tượng tranh chấp tài nguyên [4], khi hai hoặc nhiều luồng truy cập và thay đổi cùng một biến mà không có sự đồng bộ thích hợp, dẫn đến các hành vi không mong muốn hoặc không dự đoán trước được. Hiện tượng khoá chết [5] cũng là một vấn đề quan trọng trong kiểm chứng chương trình đa luồng. Khoá chết xảy ra khi hai hoặc nhiều luồng bị khoá lẫn nhau vì mỗi luồng chờ tài nguyên mà luồng khác đang giữ. Điều này làm cho chương trình ngừng hoạt động và không thể tiếp tục thực thi, gây ra tình trạng treo hệ thống. Việc phát hiện và khắc phục khoá chết là một công việc phức tạp và yêu cầu các phương pháp kiểm chứng chính thức. Khả năng không xác định (non-determinism) trong chương trình đa luồng làm tăng độ khó trong kiểm chứng [17]. Do các luồng có thể thực thi theo nhiều thứ tự khác nhau, việc kiểm thử thông thường không thể bao phủ hết các trường hợp có thể xảy ra. Điều này đòi hỏi các phương pháp kiểm chứng phải đảm bảo kiểm tra được mọi khả năng thực thi của chương trình. Một vấn đề khác là hiệu năng của các công cụ kiểm chứng. Các phương pháp như kiểm chứng mô hình thường gặp phải hiện tượng bùng nổ trạng thái, khiến cho việc kiểm chứng trở nên chậm chạp và tốn nhiều tài nguyên hơn. Để giải quyết vấn đề này, các phương pháp kiểm chứng cần phải tối ưu hóa hiệu năng và giảm bớt sự phức tạp trong quá trình kiểm chứng.

2.3 Các nghiên cứu và công cụ về kiểm chứng chương trình đa luồng

Trong những năm gần đây, nhiều nghiên cứu đã được tiến hành để giải quyết các vấn đề trong kiểm chứng chương trình đa luồng. Một số phương pháp kiểm chứng nổi bật gần đây có thể kể đến như kiểm chứng dựa trên mô hình, kiểm chứng mô

hình kết hợp với tinh chỉnh trừu tượng, kiểm chứng mô hình kết hợp với giảm thứ tự từng phần, kiểm chứng dựa trên thực thi tượng trưng, và kiểm chứng dựa trên phân tích động.

Kiểm chứng dựa trên mô hình [9] là một phương pháp kiểm chứng chương trình đa luồng dựa trên việc mô hình hóa chương trình thành một mô hình toán học và kiểm chứng tính đúng đắn của mô hình đó. Phương pháp này thường sử dụng các công cụ kiểm chứng mô hình để kiểm chứng tính đúng đắn của mô hình. Một trong những ưu điểm của kiểm chứng dựa trên mô hình là khả năng kiểm chứng chương trình lớn hoặc chương trình chứa các phần tử không xác định. Tuy nhiên, phương pháp này thường tốn nhiều thời gian và công sức hơn so với phân tích tĩnh và phân tích động. Một công cụ được phát triển từ kiểm chứng dựa trên mô hình là CBMC (C Bounded Model Checker) [11], một công cụ kiểm chứng mô hình cho chương trình C. CBMC sử dụng một mô hình toán học của chương trình để kiểm chứng tính đúng đắn của chương trình. Công cụ này hỗ trợ nhiều tính năng như kiểm chứng an toàn, kiểm chứng chạy đúng, kiểm chứng chia sẻ tài nguyên, và kiểm chứng mảng. CBMC đã được sử dụng rộng rãi trong nhiều ứng dụng thực tế và đã chứng minh được tính hiệu quả của mình trong việc kiểm chứng chương trình đa luồng. Tuy vậy, nó vẫn gặp khó khăn trong việc xử lý các chương trình lớn và chương trình chứa nhiều phần tử không xác định.

Kiểm chứng sử dụng kỹ thuật giảm thứ tự từng phần (partial order reduction) [18] là một phương pháp kiểm chứng chương trình đa luồng dựa trên việc giảm số lượng trạng thái cần kiểm tra. Phương pháp này loại bỏ các trạng thái trùng lặp và giảm số lượng trạng thái cần kiểm tra, giúp tăng tốc quá trình kiểm chứng. Tuy nhiên, phương pháp này vẫn còn một số hạn chế như không thể áp dụng cho tất cả các trường hợp và không thể giảm bớt số lượng trạng thái cần kiểm tra đến mức tối ưu. Một số công cụ kiểm chứng như Nidhugg [19], CHESS [20], và Concuerror [21] đã sử dụng kỹ thuật giảm thứ tự từng phần để tăng hiệu suất của quá trình kiểm chứng. Điểm hạn chế của những công cụ này là chúng không thể kiểm chứng được tất cả các trường hợp có thể xảy ra trong chương trình đa luồng như đã đề cập ở phần phương pháp.

Kiểm chứng dựa trên thực thi tượng trưng [10] là một phương pháp kiểm chứng chương trình đa luồng dựa trên việc thực thi tượng trưng chương trình. Phương pháp này sử dụng các công cụ kiểm chứng mô hình để mô hình hóa chương

trình và sử dụng các công cụ phân tích động để thực thi tượng trưng chương trình. Kiểm chứng dựa trên thực thi tượng trưng thường phát hiện được nhiều lỗi hơn so với kiểm chứng mô hình vì có thể phát hiện được các lỗi phát sinh do dữ liệu đầu vào hoặc các lỗi xảy ra trong quá trình thực thi. Một số công cụ kiểm chứng như KLEE [14], S2E [22] đã sử dụng kĩ thuật thực thi tượng trưng để kiểm chứng chương trình đa luồng. Tuy nhiên, những công cụ này vẫn còn một số hạn chế như không thể kiểm chứng được tất cả các trường hợp có thể xảy ra trong chương trình đa luồng.

Kiểm chứng dựa trên phân tích động [8] là một phương pháp kiểm chứng chương trình đa luồng dựa trên việc thực thi chương trình với các bộ dữ liệu đầu vào khác nhau để phát hiện lỗi. Phương pháp này thường sử dụng các công cụ kiểm thử tự động để tạo ra các bộ dữ liệu đầu vào và thực thi chương trình với các bộ dữ liệu đó. Kiểm chứng dựa trên phân tích động thường phát hiện được nhiều lỗi hơn so với kiểm chứng mô hình vì có thể phát hiện được các lỗi phát sinh do dữ liệu đầu vào hoặc các lỗi xảy ra trong quá trình thực thi. Một số công cụ kiểm chứng như Valgrind [23], Helgrind [24], và ThreadSanitizer [13] đã sử dụng kĩ thuật phân tích động để kiểm chứng chương trình đa luồng. Tuy nhiên, những công cụ này vẫn còn một số hạn chế như không thể kiểm chứng được tất cả các trường hợp có thể xảy ra trong chương trình đa luồng.

Để phát triển công cụ kiểm chứng chương trình đa luồng hiệu quả, chúng tôi kết hợp các phương pháp kiểm chứng khác nhau như kiểm chứng mô hình, kiểm chứng dựa trên thực thi tượng trưng, và kiểm chứng dựa trên phân tích động. Trong khoá luận này, chúng tôi sẽ kết hợp kiểm chứng dựa trên thực thi tượng trưng và bổ sung thêm các ràng buộc giải quyết xung đột đọc - ghi trong chương trình đa luồng. Phương pháp đề xuất sẽ giúp giảm bớt số lượng trạng thái cần kiểm tra và tăng hiệu suất của quá trình kiểm chứng.

2.4 Bộ dữ liệu kiểm chứng SV-COMP

SV-COMP (Software Verification Competition) [25] là một cuộc thi uy tín diễn ra hàng năm, tập trung vào việc đánh giá các công cụ kiểm chứng phần mềm nhằm xác định công cụ có khả năng giải quyết nhiều vấn đề nhất trong thời gian ngắn nhất. Mục tiêu của cuộc thi là tạo ra một bộ dữ liệu kiểm chứng đa dạng

và phong phú để đánh giá hiệu suất của các công cụ kiểm chứng. Bộ dữ liệu kiểm chứng SV-COMP bao gồm nhiều tập dữ liệu khác nhau như tập dữ liệu *pthread*, *pthread-nondet*, *pthread-deagle*, và *pthread-atomic*. Mỗi tập dữ liệu chứa nhiều bài toán kiểm chứng được viết bằng ngôn ngữ C và được sử dụng để đánh giá hiệu suất của các công cụ kiểm chứng. Trong khoá luận này, chúng tôi sẽ sử dụng bộ dữ liệu kiểm chứng SV-COMP để đánh giá hiệu suất của phương pháp đề xuất. Mỗi bài toán bao gồm mã nguồn và các điều kiện cần kiểm tra do người dùng cung cấp. Kết quả đầu ra của mỗi bài toán có thể thuộc một trong ba trường hợp: *True* nếu điều kiện của người dùng được thỏa mãn, *False* nếu điều kiện của người dùng bị vi phạm và tạo ra một trường hợp phản chứng, và *Timeout* nếu công cụ kiểm chứng không thể đưa ra kết quả trong thời gian quy định. Các kết quả kì vọng của các bài toán đã được xác định trước và được sử dụng để đánh giá hiệu suất của công cụ kiểm chứng.

Tập dữ liệu *pthread* [26] chứa 45 bài toán kiểm chứng tính đúng đắn của các chương trình sử dụng thư viện Pthreads để quản lý các luồng. Các chương trình trong tập này thường kiểm tra các lỗi liên quan đến tranh chấp tài nguyên, khoá chết và các vấn đề đồng bộ khác. Một số chủ đề trong tập dữ liệu này tiêu biểu như *bigshot**, *sigma**, *singleton** được đăng lên bởi dự án CSeq, *triangular** của dự án CPACheck và *The rest* được gửi từ dự án ESBMC.

Tập dữ liệu *pthread-nondet* [27] mở rộng các bài toán từ tập dữ liệu *pthread* bằng cách thêm các phần tử không xác định vào chương trình. Các phần tử không xác định này thường là các giá trị không xác định hoặc các điều kiện không xác định trong chương trình. Các bài toán trong tập này phức tạp hơn vì chúng bao gồm các tình huống mà hành vi của chương trình có thể thay đổi không xác định, đòi hỏi công cụ kiểm chứng phải có khả năng xử lý các đường thực thi có thể thay đổi theo nhiều cách khác nhau. Tập dữ liệu này được đề xuất bởi ASER Group, Parasol Lab và Đại học Texas A&M, bao gồm 6 bài toán kiểm chứng với các phần tử không xác định.

Tập dữ liệu *pthread-deagle* [28] bao gồm các bài toán kiểm chứng phức tạp hơn, yêu cầu kiểm tra các tính chất an toàn và bảo mật của các chương trình đa luồng. Các bài toán này thường bao gồm các điều kiện tiền điều kiện và hậu điều kiện phức tạp, đòi hỏi công cụ kiểm chứng phải có khả năng phân tích và xác minh chính xác các điều kiện này. Tập dữ liệu này được đề xuất bởi dự án Deagle, một công cụ kiểm

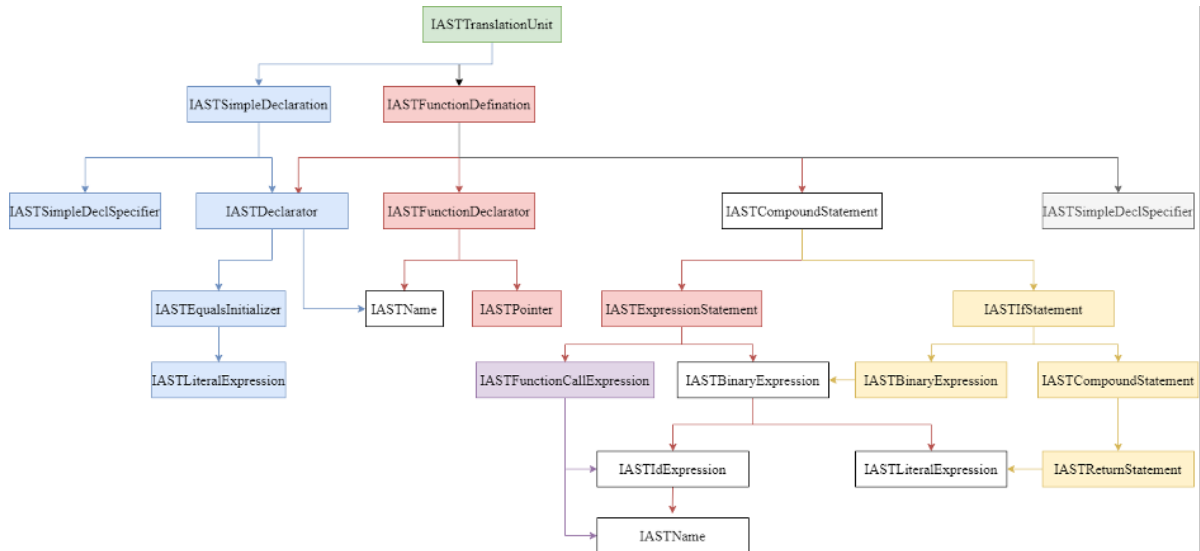
chứng chương trình đa luồng dựa lý thuyết nhất quán sắp xếp. Sau đó, nó được điều chỉnh và sửa đổi bởi nhiều dự án khác nhau. Với tập `airline` đến từ dự án JMCR và sau đó được điều chỉnh bởi dự án Nidhugg. Tập `floating_red` được đề xuất bởi dự án Nidhugg năm 2018 và các tập `arithmetic_prog`, `circular_buffer`, `reorder_c11` được điều chỉnh bởi dự án SCTBench.

Tập dữ liệu *pthread-atomic* [29] tập trung vào các bài toán kiểm chứng liên quan đến các thao tác nguyên tử (atomic operations) trong các chương trình đa luồng. Các chương trình trong tập này thường sử dụng các thao tác nguyên tử để đảm bảo tính toàn vẹn của dữ liệu và tránh các lỗi liên quan đến tranh chấp tài nguyên.

2.5 Cây cú pháp trừu tượng

Như đã trình bày ở phần trước, khoá luận này sẽ tập trung vào nghiên cứu về kiểm chứng chương trình đa luồng dựa trên thực thi tượng trưng. Để thực hiện việc này, chúng tôi cần mô hình hóa chương trình thành một cấu trúc dữ liệu phù hợp để thực thi tượng trưng chương trình. Trong lĩnh vực kiểm chứng phần mềm, cây cú pháp trừu tượng (Abstract Syntax Tree - AST) là một cấu trúc dữ liệu phổ biến được sử dụng để mô hình hóa chương trình. AST là một biểu diễn cấu trúc của chương trình dưới dạng cây, trong đó mỗi nút của cây tương ứng với một phần tử trong chương trình. AST thường được sử dụng trong các công cụ phân tích mã nguồn để phân tích cú pháp và cấu trúc của chương trình.

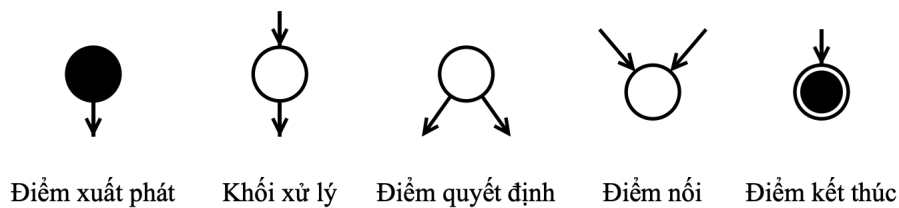
Hình 2.2 minh họa kiến trúc của cây cú pháp trừu tượng. Cây cú pháp trừu tượng bao gồm nhiều nút, mỗi nút tương ứng với một phần tử trong chương trình. Các nút trong cây cú pháp trừu tượng thường được phân loại thành các loại khác nhau như nút biểu thức, nút câu lệnh, nút khai báo, nút hàm, nút biến, nút hằng số, và nút toán tử. Các nút trong cây cú pháp trừu tượng thường chứa các thuộc tính như tên, kiểu dữ liệu, giá trị, và các nút con. Các nút con của một nút thường tương ứng với các phần tử con của phần tử đó trong chương trình. Cây cú pháp trừu tượng sẽ là đầu vào cho quá trình xây dựng đồ thị luồng điều khiển sẽ được đề cập ở phần sau.



Hình 2.2: Kiến trúc của cây cú pháp trừu tượng

2.6 Đồ thị luồng điều khiển

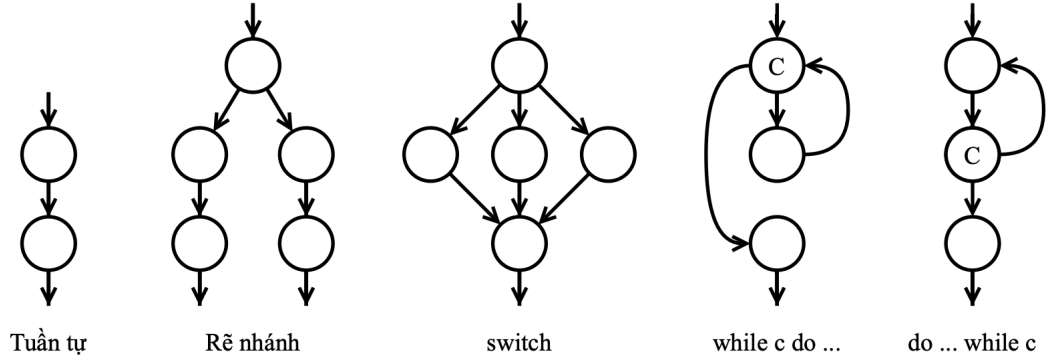
Đồ thị luồng điều khiển (Control Flow Graph - CFG) là một cấu trúc dữ liệu được sử dụng để biểu diễn các luồng điều khiển trong một chương trình máy tính. Mỗi nút trong CFG đại diện cho một khối cơ bản của chương trình, và mỗi cạnh đại diện cho các đường đi có thể có giữa các khối cơ bản này khi chương trình thực thi. CFG là một công cụ quan trọng trong việc phân tích và tối ưu hóa mã nguồn, đặc biệt là trong các trình biên dịch và các công cụ kiểm chứng phần mềm.



Hình 2.3: Các thành phần cơ bản trong đồ thị luồng điều khiển

Hình 2.3 minh họa các thành phần cơ bản trong đồ thị luồng điều khiển. Điểm xuất phát biểu thị cho điểm bắt đầu, điểm kết thúc biểu thị cho điểm kết thúc, các khối xử lý biểu thị cho các khối cơ bản của chương trình, các điểm quyết định đại diện cho các điểm mà chương trình tạo ra một luồng mới, mỗi điểm này có thể có nhiều nhánh, mỗi nhánh tương ứng với một luồng mới được tạo ra. Điểm nối biểu thị các điểm mà chương trình chờ đợi tất cả các luồng con kết thúc. Mỗi

điểm nối có thể có nhiều nhánh đến, mỗi nhánh tương ứng với một luồng con. Các cạnh trong đồ thị luồng điều khiển biểu diễn các đường đi có thể có giữa các khối cơ bản trong chương trình. Mỗi cạnh trong CFG biểu diễn một đường đi có thể có giữa hai khối cơ bản, và mỗi đường đi trong CFG tương ứng với một trình tự thực thi của chương trình.



Hình 2.4: Các cấu trúc điều khiển cơ bản trong đồ thị luồng điều khiển

Hình 2.4 minh họa các cấu trúc điều khiển cơ bản trong đồ thị luồng điều khiển. Cấu trúc tuần tự biểu diễn một chuỗi các khối cơ bản được thực thi theo trình tự. Cấu trúc rẽ nhánh biểu diễn một điểm quyết định mà chương trình chọn một trong nhiều đường đi để thực thi. Cấu trúc lặp biểu diễn một vòng lặp mà chương trình thực thi một chuỗi các khối cơ bản nhiều lần. Cấu trúc rẽ nhánh đa điều kiện (switch) biểu diễn một điểm quyết định mà chương trình chọn một trong nhiều đường đi để thực thi dựa trên một điều kiện.

2.7 Bộ giải SMT

Bộ giải SMT (Satisfiability Modulo Theories) [30] là một công cụ quan trọng trong lĩnh vực kiểm chứng chương trình, đặc biệt là trong thực thi tượng trưng. SMT là sự mở rộng của SAT (Boolean Satisfiability Problem), nơi các biến không chỉ là biến Boolean mà còn thuộc nhiều loại khác nhau như số nguyên, số thực, và các cấu trúc dữ liệu phức tạp hơn. SAT là bài toán xác định liệu có một cách gán giá trị cho các biến Boolean sao cho công thức logic là đúng. SMT mở rộng SAT bằng cách thêm vào các lý thuyết (theories) như lý thuyết số học, lý thuyết về mảng, và lý thuyết về dữ liệu có cấu trúc. Một bộ giải SMT cố gắng tìm một cách gán giá trị cho các biến sao cho công thức, bao gồm các ràng buộc từ các lý thuyết

này, được thỏa mãn. Trong kiểm chứng chương trình, bộ giải SMT thường được sử dụng để giải quyết các ràng buộc từ việc thực thi tượng trưng chương trình. Bộ giải SMT giúp xác định các giá trị của các biến sao cho các ràng buộc được thỏa mãn, giúp phát hiện lỗi và xác minh tính đúng đắn của chương trình. Ở khoá luận này, chúng tôi sử dụng bộ giải SMT Z3 [30], một trong những bộ giải SMT phổ biến và mạnh mẽ nhất hiện nay, để giải quyết các ràng buộc trong quá trình thực thi tượng trưng chương trình.

Chương 3

Phương pháp giải quyết xung đột trong kiểm chứng chương trình đa luồng

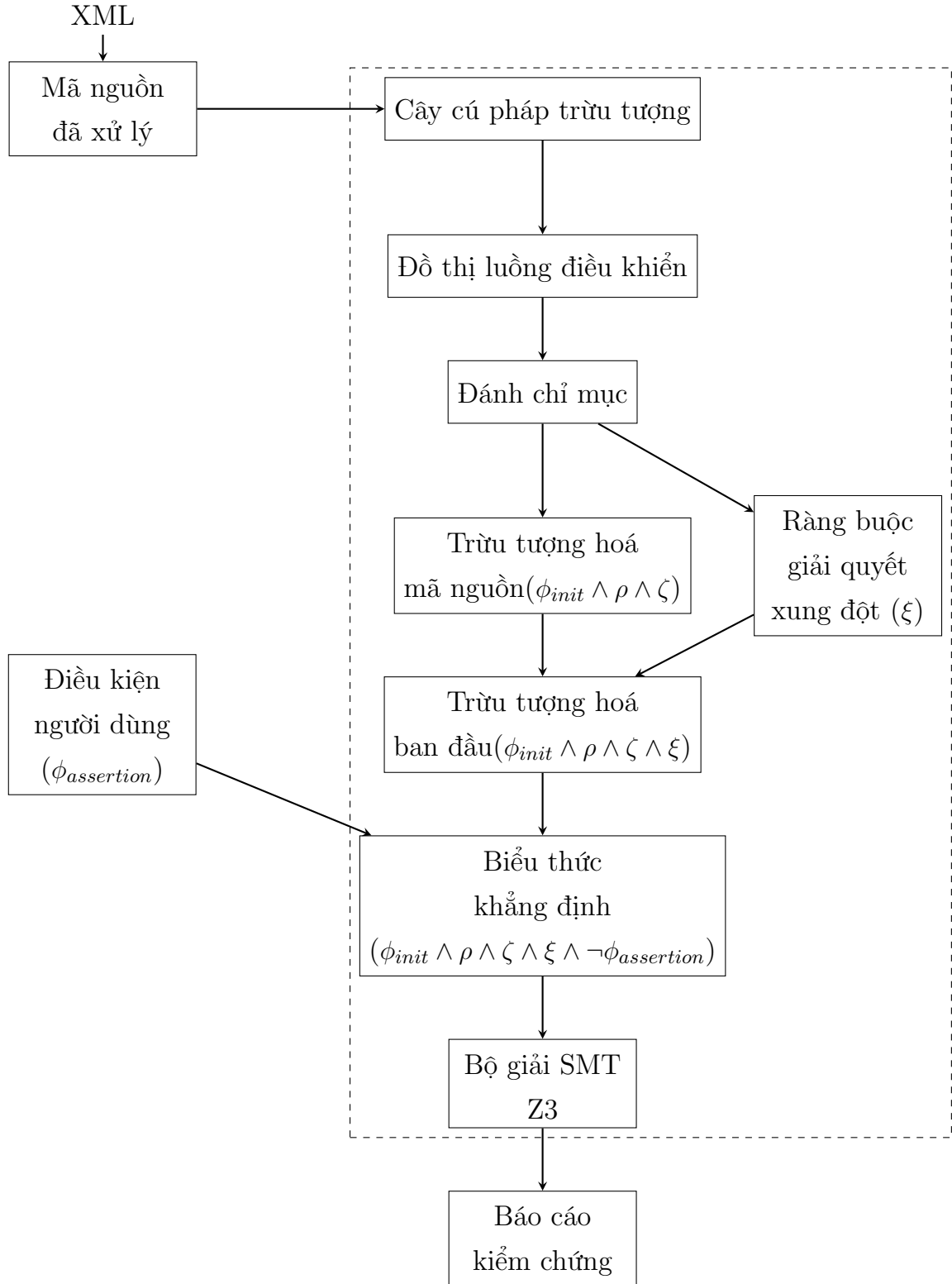
3.1 Tổng quan về phương pháp đề xuất

Qua trình bày ở các chương trước cho thấy, việc kiểm chứng các chương trình đa luồng vốn dĩ đã là một thách thức do sự phức tạp gây ra bởi tính thực thi đồng thời. Các luồng có thể tương tác với nhau theo những cách không thể đoán trước, dẫn đến hiện tượng race condition, deadlock và các vấn đề về đồng thời khác. Các phương pháp truyền thống để kiểm chứng các chương trình như vậy thường gặp phải vấn đề bùng nổ trạng thái, trong đó số lượng trạng thái có thể có của chương trình tăng theo cấp số nhân với số lượng luồng và biến.

Để giảm thiểu vấn đề này, khoá luận đề xuất một phương pháp dựa trên kỹ thuật thực thi tượng trưng, để kiểm chứng các chương trình C/C++ đa luồng. Nhờ tận dụng thực thi tượng trưng thay vì giá trị cụ thể, phương pháp này khám phá một cách có hệ thống các đường thực thi tiềm năng và phát hiện các vi phạm điều kiện khẳng định. Nó tạo điều kiện cho việc phân tích nhiều đường dẫn thực thi đồng thời và giúp trừu tượng hóa hành vi của chương trình mà không cần phải liệt kê rõ ràng từng trạng thái có thể xảy ra.

Hình 3.1 minh họa kiến trúc tổng quan của phương pháp đề xuất. Quy trình làm việc bắt đầu với việc nhập mã nguồn C/C++, để loại bỏ sự phức tạp của mã nguồn, mã nguồn được định dạng trước khi được đưa vào quá trình phân tích. Đầu vào của chương trình còn có một tệp XML lưu trữ thông tin của mã nguồn, trong đó quy định tính chất mà chương trình cần phải thỏa mãn. Sau đó, mã nguồn được chuyển đổi thành cây cú pháp trừu tượng (AST), biểu diễn cấu trúc cú pháp của chương trình. AST cung cấp một cách biểu diễn phân cấp về cấu trúc cú pháp của chương trình, làm nền tảng cho các phân tích tiếp theo.

AST được chuyển đổi thành đồ thị luồng điều khiển (CFG), biểu diễn các luồng điều khiển có thể xảy ra trong chương trình. CFG bao gồm các nút (biểu



Hình 3.1: Kiến trúc tổng quan của phương pháp kiểm chứng chương trình đa luồng dựa trên kỹ thuật thực thi tượng trưng

diễn các câu lệnh của chương trình) và các cạnh (biểu diễn các chuyển đổi luồng điều khiển). Để xử lý các vấn đề liên quan đến đồng thời, CFG được bổ sung với thông tin chỉ mục, rất quan trọng để theo dõi thứ tự của các hoạt động đọc và ghi trên biến chia sẻ. Việc chỉ mục này là cần thiết để xác định các vấn đề liên quan đến đồng thời tiềm ẩn và các lỗi liên quan đến đồng thời khác.

CFG được lập chỉ mục sau đó được trừu tượng hóa thành một công thức logic bậc nhất (FOL). Sự trừu tượng hóa này nắm bắt hành vi của chương trình dưới dạng các vị từ và quan hệ logic, tập trung vào các tương tác giữa các luồng và các biến được chia sẻ. Sự trừu tượng ban đầu bao gồm trạng thái ban đầu (ϕ_{init}), các chuyển đổi luồng (ρ), và ràng buộc (ζ) biểu diễn logic và luồng điều khiển của chương trình.

Các hoạt động đọc và ghi đồng thời trên biến chia sẻ có thể dẫn đến xung đột. Phương pháp này tích hợp một cơ chế để giải quyết các xung đột này và đảm bảo một cái nhìn nhất quán về bộ nhớ. Quá trình giải quyết được biểu diễn bằng công thức ξ , được tích hợp vào trừu tượng ban đầu. Bằng cách kết hợp trừu tượng FOL của hành vi của chương trình với cơ chế giải quyết xung đột, trừu tượng ban đầu ($\phi_{init} \wedge \rho \wedge \zeta \wedge \xi$) được hình thành. Trừu tượng này phục vụ như điểm khởi đầu cho quá trình kiểm chứng. Người dùng cung cấp một khẳng định ($\phi_{assertion}$) chỉ định tính chất hoặc điều kiện mà họ muốn kiểm chứng trong chương trình. Khẳng định này thường liên quan đến tính an toàn hoặc tính sống.

Trừu tượng ban đầu và khẳng định của người dùng được kết hợp thành công thức khẳng định ($\phi_{init} \wedge \rho \wedge \zeta \wedge \xi \wedge \neg \phi_{assertion}$). Công thức này biểu diễn phủ định của khẳng định của người dùng, cho phép công cụ tìm kiếm các trường hợp phản chứng vi phạm khẳng định. Công thức khẳng định sau đó được đưa vào một bộ giải SMT (Satisfiability Modulo Theories), chẳng hạn như Z3. Bộ giải SMT sẽ xác định xem công thức khẳng định có thể được thỏa mãn hay không. Nếu bộ giải SMT phát hiện công thức khẳng định không thể thỏa mãn, điều đó có nghĩa là chương trình thỏa mãn khẳng định của người dùng dưới tất cả các cách xen kẽ có thể của các luồng. Nếu công thức có thể thỏa mãn, nó cho thấy rằng tồn tại một trường hợp phản chứng mà khẳng định bị vi phạm, và công cụ sẽ tạo ra một báo cáo kiểm chứng nêu bật lỗi tiềm ẩn này.

Để hiểu rõ hơn về phương pháp đề xuất, các phần tiếp theo của chương này sẽ trình bày chi tiết về các pha trong phương pháp này.

3.2 Trừu tượng hóa chương trình đa luồng dựa trên kỹ thuật thực thi tượng trưng

3.2.1 Pha xây dựng cây cú pháp trừu tượng

Như đã trình bày trong phần 3.1, việc xây dựng cây cú pháp trừu tượng (AST) từ mã nguồn đầu vào là một bước quan trọng trong phương pháp được đề xuất để kiểm chứng các chương trình đa luồng. AST biểu diễn cấu trúc cú pháp phân cấp của mã nguồn, đóng vai trò là nền tảng cho các phân tích và quy trình kiểm chứng tiếp theo như thực thi tượng trưng. Phần này trình bày chi tiết quá trình xây dựng AST từ mã nguồn đầu vào bằng cách sử dụng thư viện Eclipse CDT (C/C++ Development Tooling) và các cân nhắc cụ thể cho các chương trình đa luồng. Eclipse CDT cung cấp các công cụ mạnh mẽ để phân tích và phân tích mã nguồn C/C++. Quá trình xây dựng AST bằng cách sử dụng thư viện Eclipse CDT được thể hiện trong Thuật toán 1.

Thuật toán 1: Xây dựng cây cú pháp trừu tượng từ mã nguồn đầu vào

Đầu vào: filelocation - đường dẫn đến tệp mã nguồn

Đầu ra: translationUnit - AST của mã nguồn

1 Function createAST(*filelocation*):

```
2   fileContent ← FileContent.createForExternalFileLocation(filelocation);
3   includeFile ← IncludeFileContentProvider.getEmptyFilesProvider();
4   log ← new DefaultLogService();
5   includePaths ← new String[0];
6   info ← new ScannerInfo(new HashMap<String, String>(),
      includePaths);
7   translationUnit ←
      GPPLanguage.getDefault().getASTTranslationUnit(fileContent, info,
      includeFile, null, 0, log);
8   return translationUnit;
```

Ban đầu, mã nguồn trải qua quá trình phân tích từ vựng, trong đó nó được mã hóa thành các thành phần có thể quản lý được. Các mã thông báo này sau đó được chuyển qua giai đoạn phân tích cú pháp, sắp xếp chúng thành một cây phân tích cú pháp đại diện cho cấu trúc cú pháp của chương trình. Cây phân tích cú pháp sau đó được chuyển đổi thành AST, một biểu diễn phân cấp trừu tượng

hóa các chi tiết trong khi vẫn giữ lại cấu trúc cần thiết cho phân tích sâu hơn. Quá trình chuyển đổi này bao gồm việc xác định các cấu trúc cụ thể như hàm luồng, nguyên hàm đồng bộ hóa và các biến được chia sẻ, rất quan trọng trong các chương trình đa luồng. Mỗi cấu trúc được đóng gói trong các nút AST tương ứng, sau đó được tích hợp vào cây rộng hơn. Bằng cách tận dụng thư viện Eclipse CDT, phương pháp này phân tích cú pháp và xây dựng AST một cách hiệu quả, đảm bảo biểu diễn chính xác cả các phần tử chương trình tiêu chuẩn và đa luồng. Biểu diễn có cấu trúc này là cơ bản cho các bước tiếp theo trong kiểm chứng chương trình, chẳng hạn như thực thi tượng trưng và phân tích điều kiện đường dẫn.

3.2.2 Pha xây dựng đồ thị luồng điều khiển

Pha xây dựng đồ thị luồng điều khiển là một bước quan trọng trong quy trình kiểm chứng chương trình đa luồng. Đồ thị luồng điều khiển là một biểu diễn đồ thị của tất cả các đường dẫn mà chương trình có thể thực thi. Mỗi nút trong đồ thị đại diện cho một lệnh hoặc một khối lệnh trong chương trình, trong khi các cạnh đại diện cho sự chuyển đổi luồng điều khiển giữa các lệnh đó. Việc xây dựng đồ thị luồng điều khiển giúp chúng ta hiểu rõ hơn về cấu trúc của chương trình, từ đó dễ dàng hơn trong việc phân tích và kiểm chứng chương trình.

Thuật toán 2 mô tả quá trình xây dựng đồ thị luồng điều khiển từ AST. Đầu vào của thuật toán là một câu lệnh trong chương trình, đầu ra là đồ thị luồng điều khiển tương ứng với câu lệnh đó. Trong quá trình xây dựng, thuật toán kiểm tra loại câu lệnh và tạo các nút tương ứng trong đồ thị luồng điều khiển. Cụ thể, nếu câu lệnh là *CompoundStatement*, thuật toán sẽ tạo một đồ thị con cho mỗi câu lệnh con trong *CompoundStatement* và nối chúng lại với nhau. Nếu câu lệnh là *IfStatement*, *ReturnStatement*, hoặc *DeclarationStatement*, thuật toán sẽ tạo các nút tương ứng trong đồ thị luồng điều khiển. Trong trường hợp còn lại, thuật toán tạo một nút đơn giản (plain node) trong đồ thị luồng điều khiển.

3.2.3 Pha đánh chỉ mục biến

Pha đánh chỉ mục biến là bước tiếp theo trong quá trình kiểm chứng chương trình đa luồng, giúp theo dõi sự truy cập và cập nhật của các biến trong các luồng khác nhau. Mỗi biến được gán một chỉ mục duy nhất để xác định chính xác vị trí và

Thuật toán 2: Xây dựng đồ thị luồng điều khiển từ AST

Đầu vào: *statement* - câu lệnh cần xây dựng

Đầu ra: *cfg* - đồ thị luồng điều khiển

```
1 Function createSubGraph(statement):  
2   cfg  $\leftarrow$  new ControlFlowGraph();  
3   if statement is CompoundStatement then  
4     stmts  $\leftarrow$  getStatements(statement);  
5     for stmt in stmts do  
6       subCFG  $\leftarrow$  createSubGraph(stmt);  
7       if subCFG and cfg are not null then  
8         cfg.concat(subCFG);  
9       end  
10    end  
11  end  
12  else if statement is IfStatement then  
13    cfg  $\leftarrow$  createIf(statement);  
14  end  
15  else if statement is ReturnStatement then  
16    returnNode  $\leftarrow$  new ReturnNode(statement);  
17    cfg  $\leftarrow$  new ControlFlowGraph(returnNode, returnNode);  
18  end  
19  else if statement is DeclarationStatement then  
20    cfg  $\leftarrow$  createDeclaration(statement);  
21  end  
22  else  
23    plainNode  $\leftarrow$  new PlainNode(statement);  
24    cfg  $\leftarrow$  new ControlFlowGraph(plainNode, plainNode);  
25  end  
26  return cfg;
```

thứ tự truy cập của nó trong chương trình. Mỗi biến trong chương trình được gán một chỉ mục ban đầu. Chỉ mục này được thiết lập dựa trên ngữ cảnh của biến, bao gồm vị trí khai báo và luồng thực thi. Các biến được phân loại thành biến đọc (read) và biến ghi (write). Mỗi lần một biến được truy cập trong một luồng, một chỉ mục mới được gán cho biến đó để phản ánh thứ tự truy cập. Các biến đọc được đánh chỉ mục bằng ký hiệu X_{rij} và biến ghi được đánh chỉ mục bằng ký hiệu X_{wij} , trong đó:

- r, w : Chỉ mục biểu thị các biến đọc và ghi tương ứng.
- i : Chỉ mục cho biết luồng của biến. Đối với luồng khởi tạo, ta có $i = 0$. Khi biến xuất hiện trong câu lệnh kiểm chứng chương trình, giả định $i = n + 1$.
- j : Thứ tự xuất hiện của một biến trong mỗi luồng.

Ví dụ, với một biến được đánh chỉ mục như X_{r01} , điều này có nghĩa là biến X được đọc từ bộ nhớ và được khởi tạo tại thời điểm khai báo. Các chỉ mục 0 và 1 chỉ ra rằng đây là hoạt động đọc ban đầu cho biến này trong luồng đầu tiên của chương trình. Thuật toán 3 mô tả quá trình đánh chỉ mục biến trong đồ thị luồng điều khiển. Đầu vào của thuật toán là nút bắt đầu của đồ thị luồng điều khiển và quản lý biến, đầu ra là các biến được đánh chỉ mục trong đồ thị luồng điều khiển. Trong quá trình thực thi, thuật toán duyệt qua các nút trong đồ thị luồng điều khiển và đánh chỉ mục các biến dựa trên ngữ cảnh của chúng. Cụ thể, thuật toán xác định các nút quyết định, nút bắt đầu, nút kết thúc điều kiện, nút không đổi và nút thông thường trong đồ thị luồng điều khiển và đánh chỉ mục các biến tương ứng. Nếu nút là nút bắt đầu hoặc nút kết thúc, thuật toán sẽ duyệt qua các nút liên quan và đánh chỉ mục các biến trong chúng. Nếu nút là nút quyết định, thuật toán sẽ đánh chỉ mục các biến và duyệt qua nút kết thúc của nút quyết định. Cuối cùng, thuật toán sẽ duyệt qua các nút còn lại trong đồ thị luồng điều khiển và đánh chỉ mục các biến trong chúng.

Áp dụng cách mã hoá này cho chương trình đa luồng trên Hình 2.1, ta có chương trình được mã hoá như trên Hình 3.4. Các biến i và j được mã hoá dưới dạng i_{r01} , j_{r01} , i_{w11} , j_{w21} , i_{w12} , j_{w22} , i_{w13} , j_{w23} , i_{w14} , j_{w24} , i_{w15} , j_{w25} , i_{r31} , j_{r31} thể hiện các hoạt động đọc-ghi tương ứng và thứ tự xuất hiện của các biến trong mỗi luồng. Với kết quả mã hoá như vậy, ta có thể thực thi chương trình đa luồng bằng cách sử dụng kỹ thuật thực thi tượng trưng. Hình 3.3 thể hiện Cây thực thi tượng

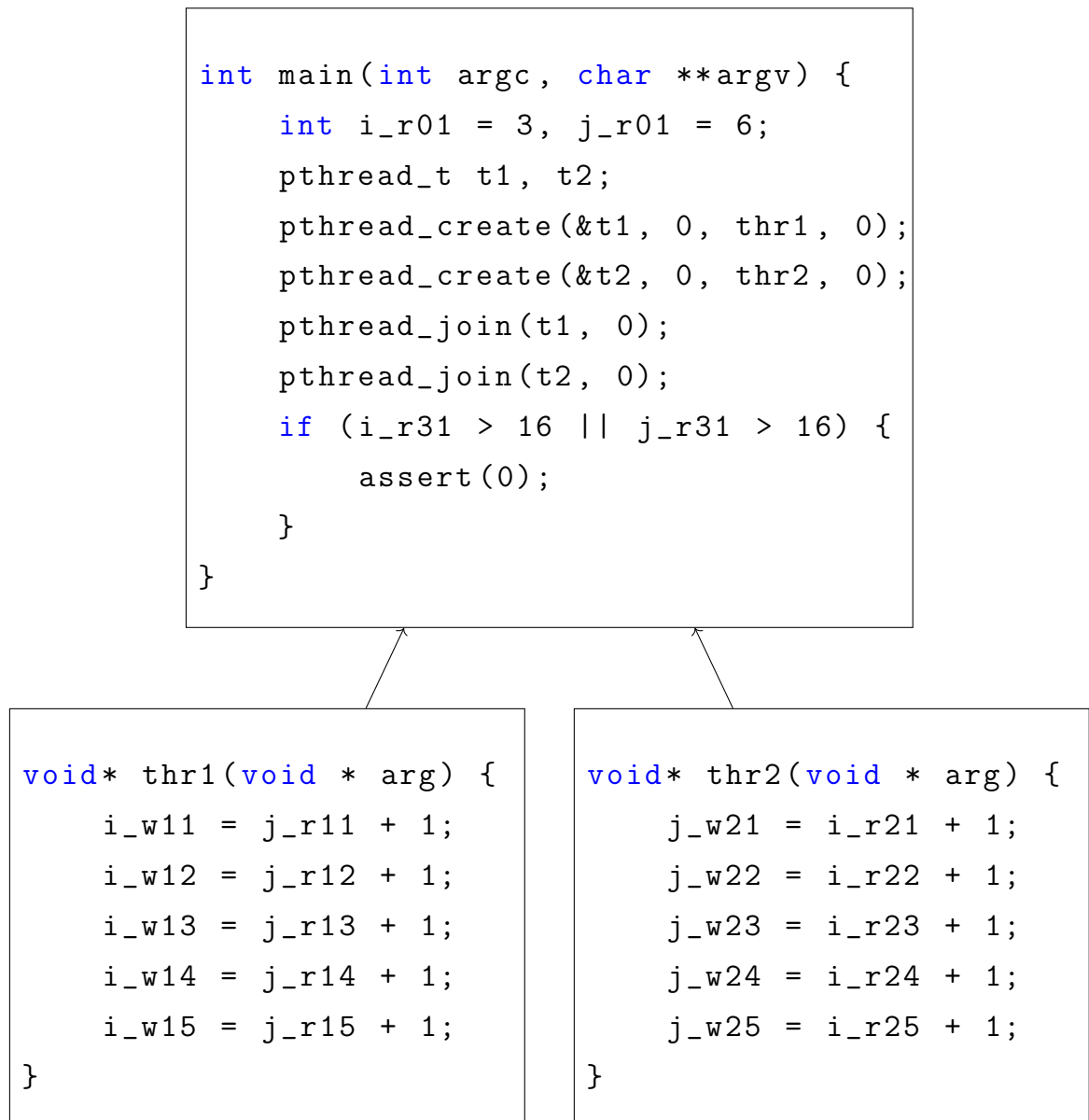
Thuật toán 3: Đánh chỉ mục biến

Đầu vào: start - nút bắt đầu của CFG, vm - quản lý biến

Đầu ra: Biến được đánh chỉ mục trong đồ thị luồng điều khiển

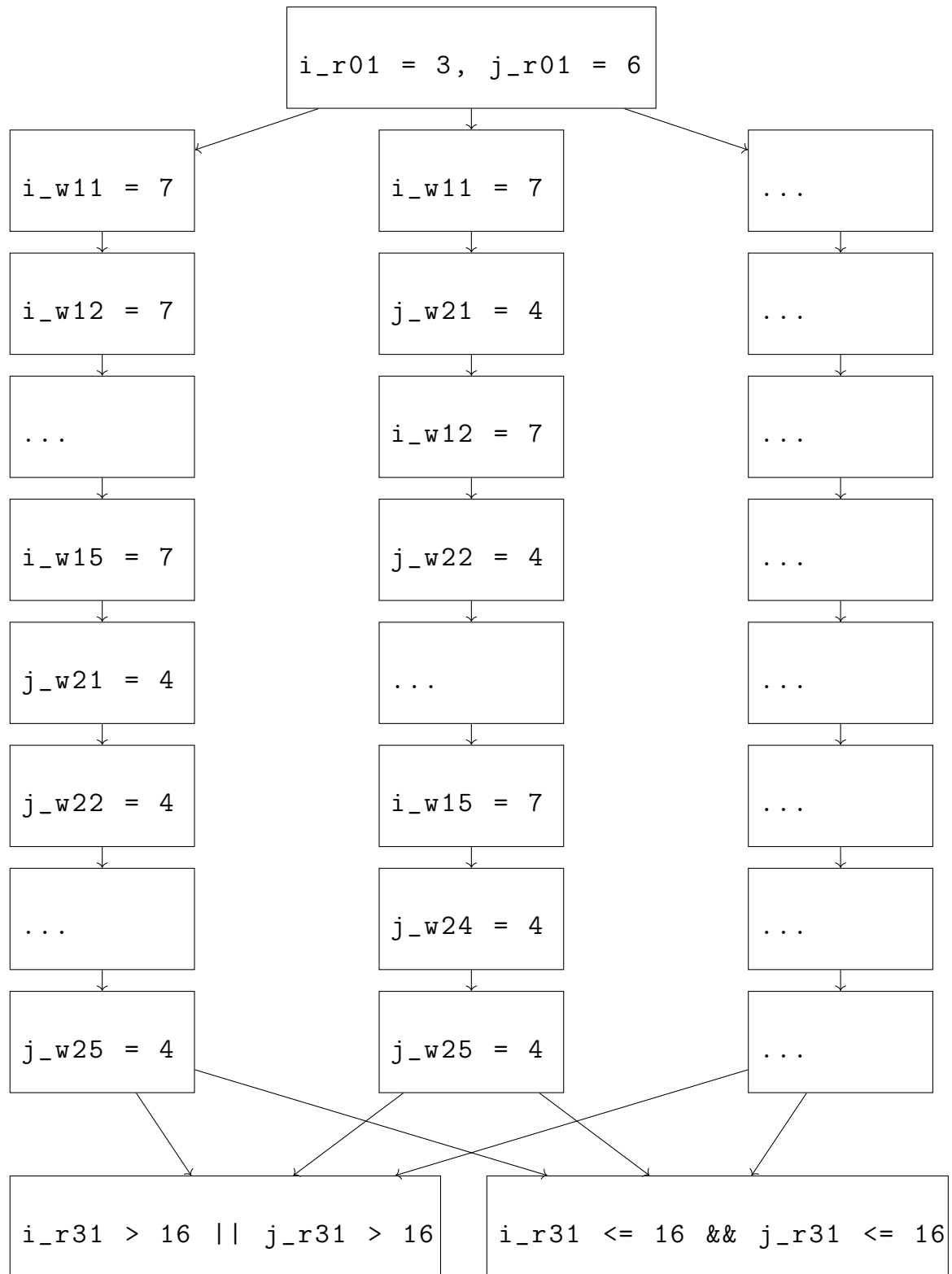
```
1 Function iteration(start, vm):
2   iter ← start;
3   else if iter is DecisionNode then
4     | iter.index(vm);
5     | iteration(iter.getEndNode());
6   end
7   else if iter is BeginNode then
8     | iter.index(vm);
9     | iteration(iter.getNext());
10    | iter.getEndNode().index(vm);
11    | iteration(iter.getEndNode().getNext());
12  end
13  else if iter is EndConditionNode then
14    | iter.index(vm);
15  end
16  else if iter is PlainNode then
17    | iter.index(vm);
18    | iteration(iter.getNext());
19  end
20  else
21    | if iter is BeginFunctionNode then
22      | vm.increaseCurrentThreadIndex();
23    | end
24    | else if iter is EndFunctionNode then
25      | vm.increaseCurrentThreadIndex();
26    | end
27    | iter.index(vm);
28    | iteration(iter.getNext());
29  end
```

trung của chương trình đa luồng đã được mã hoá. Do các giá trị của i và j được xác định bởi sự đan xen của hai luồng, có nhiều đường thực thi khả thi trong cây thực thi tương trưng, cây sẽ khá lớn do có nhiều khả năng xen kẽ các hoạt động đọc và ghi trong hai luồng. Tuy nhiên, chúng ta có thể biểu diễn một cây thực thi ký hiệu đơn giản hóa, nắm bắt được bản chất hành vi của chương trình và khả năng vi phạm điều kiện khẳng định. Trong đó:



Hình 3.2: Chương trình đa luồng được mã hoá

- **Trạng thái khởi tạo:** Cây bắt đầu với trạng thái khởi tạo, trong đó $i_{r01} = 3$ và $j_{r01} = 6$.



Hình 3.3: Cây thực thi tượng trưng của chương trình đa luồng

- **Xen kẽ luồng:** Cây phân nhánh để biểu diễn các khả năng xen kẽ khác nhau của các hoạt động đọc và ghi trong hai luồng (**thr1** và **thr2**). Mỗi nhánh biểu

diễn một thứ tự cụ thể mà các câu lệnh của hai luồng được thực thi.

- **Kiểm tra điều kiện khẳng định:** Ở cuối mỗi khả năng xen kẽ, điều kiện khẳng định ($i_{r31} > 16 || j_{r31} > 16$) được kiểm tra.
- **Vi phạm điều kiện khẳng định:** Nếu điều kiện khẳng định là đúng cho bất kỳ lần xen kẽ nào, cây sẽ đến một nút lá có nhãn "Vi phạm điều kiện khẳng định", cho biết rằng chương trình có thể vi phạm điều kiện khẳng định.
- **Không vi phạm điều kiện khẳng định:** Nếu điều kiện khẳng định là sai đối với tất cả các lần xen kẽ, chương trình được coi là an toàn đối với điều kiện khẳng định.

Ví dụ này chứng minh tính không tất định vốn có trong các chương trình đa luồng. Thứ tự chính xác mà các luồng được lên lịch có thể tác động đáng kể đến kết quả cuối cùng và việc các điều kiện khẳng định có bị vi phạm hay không. Thực thi tượng trưng giúp khám phá một cách có hệ thống các lần xen kẽ khác nhau này để đảm bảo quá trình kiểm chứng kỹ lưỡng.

Tuy nhiên, việc thay đổi giá trị của cùng một biến trong các nhánh của một câu lệnh `If else` có thể dẫn đến sự khác biệt về chỉ mục của biến sau khi kết thúc khối `If else`. Điều này khiến việc xác định chỉ mục thực sự của biến trong các câu lệnh tiếp theo trở nên khó khăn. Để giải quyết vấn đề này, chúng ta cần đồng bộ chỉ mục của các biến đã thay đổi trong khối `If else`. Đồng bộ chỉ mục được thực hiện ngay sau khi đánh chỉ mục trong cả hai nhánh của nút điều kiện và được thể hiện trong Thuật toán 4.

Trong thuật toán này, chúng ta đồng bộ chỉ mục của các biến trong các nhánh của một câu lệnh `If else`. Trước hết, nó so sánh chỉ mục của các biến trong cả hai nhánh và thực hiện các hành động tương ứng để đảm bảo rằng chỉ mục của các biến được đồng bộ hóa. Nếu chỉ mục của biến trong nhánh `Then` nhỏ hơn chỉ mục của biến trong nhánh `Else`, chúng ta thêm biến từ nhánh `Else` vào danh sách biến của nhánh `Then`. Nếu chỉ mục của biến trong nhánh `Else` nhỏ hơn chỉ mục của biến trong nhánh `Then`, chúng ta tạo một nút đồng bộ hóa và thêm nó vào cây thực thi tượng trưng.

Thuật toán 4: Đồng bộ chỉ mục biến

Đầu vào: VariableManage *vm* - đối tượng quản lý biến

Đầu ra: *thenVM* - đối tượng quản lý biến sau khi đồng bộ chỉ mục

```
1 size  $\leftarrow$  min(thenVM.getInputListSize(), elseVM.getInputListSize());
2 for i  $\leftarrow$  0 to size - 1 do
3   thenVar  $\leftarrow$  thenVM.getVar(i);
4   elseVar  $\leftarrow$  elseVM.getVar(i);
5   if thenVar.getSsaIndex() < elseVar.getSsaIndex() then
6     | thenVM.getInputList().add(elseVar);
7   end
8   else if elseVar.getSsaIndex() < thenVar.getSsaIndex() then
9     | rightHand  $\leftarrow$  elseVar.getVariableWithIndex();
10    | elseVar.setSsaIndex(thenVar.getSsaIndex());
11    | leftHand  $\leftarrow$  elseVar.getVariableWithIndex();
12    | syncNode  $\leftarrow$  new SyncNode(leftHand, rightHand);
13    | endOfElse.setNext(syncNode);
14    | syncNode.setNext(endNode);
15    | setEndOfElse(syncNode);
16  end
17 end
18 return thenVM
```

3.2.4 Pha trừu tượng hoá chương trình đầu vào

Pha trừu tượng hoá chương trình đầu vào giúp đơn giản hóa và trừu tượng hóa các chi tiết phức tạp của mã nguồn ban đầu. Mục tiêu của pha này là tạo ra một phiên bản đơn giản hơn của chương trình, giữ lại các thuộc tính quan trọng cần thiết cho việc kiểm chứng, đồng thời loại bỏ các chi tiết không cần thiết. Các chi tiết không ảnh hưởng đến hành vi tổng quát của chương trình, chẳng hạn như các lệnh in ra màn hình, các biến tạm thời không ảnh hưởng đến luồng điều khiển chính, được loại bỏ. Điều này giúp đơn giản hóa chương trình và tập trung vào các khía cạnh quan trọng cần kiểm chứng. Các cấu trúc điều khiển như vòng lặp, điều kiện rẽ nhánh được trừu tượng hóa thành các mô hình đơn giản hơn. Ví dụ, một vòng lặp có thể được thay thế bằng một mô hình trừu tượng thể hiện số lần lặp tối đa cần thiết để kiểm chứng tính đúng đắn của chương trình.

Như đã đề cập ở pha trước, các biến và câu lệnh trong chương trình được đánh chỉ mục để theo dõi chính xác thứ tự truy cập và cập nhật. Chỉ mục này cũng được duy trì trong phiên bản trừu tượng của chương trình. Một mô hình trừu tượng của chương trình được xây dựng từ các thành phần đã được đơn giản hóa và trừu tượng hóa. Mô hình này giữ lại các thuộc tính quan trọng như trạng thái biến, luồng điều khiển, và các điều kiện kiểm chứng, nhưng loại bỏ các chi tiết không cần thiết. Sự trừu tượng hoá này bao gồm một số thành phần:

$$\Phi_{\text{final}} = \phi_{\text{init}} \wedge \rho \wedge \zeta \wedge \neg\phi_{\text{assertion}} \wedge \xi \quad (3.1)$$

- **Trạng thái khởi tạo (ϕ_{init}):** Công thức này biểu diễn các giá trị ban đầu của các biến trong chương trình trước khi bất kỳ luồng nào bắt đầu thực thi. Thông thường bao gồm việc khởi tạo các biến toàn cục.
- **Chuyển tiếp luồng (ρ):** Đối với mỗi luồng trong chương trình, một công thức ρ_{thr} được tạo ra để biểu diễn các chuyển đổi giữa các trạng thái trong luồng đó. Các chuyển đổi này được xác định bởi các lệnh của luồng và giá trị của các biến cục bộ của nó. Các chuyển tiếp luồng tổng thể cho chương trình được biểu diễn bằng sự kết hợp của tất cả các chuyển tiếp luồng riêng lẻ ($\rho = \rho_{\text{thr1}} \wedge \rho_{\text{thr2}} \wedge \dots \wedge \rho_{\text{thr}n} \wedge \rho_{\text{main}}$).
- **Mối quan hệ đọc từ (ζ):** Công thức này nắm bắt mối quan hệ giữa các sự

kiện đọc và ghi trong chương trình. Nó chỉ ra rằng mỗi sự kiện đọc có khả năng đọc giá trị được ghi bởi bất kỳ sự kiện ghi nào vào cùng một biến. Đây là một sự đơn giản hóa của hành vi chương trình thực tế, vì nó bỏ qua thứ tự xảy ra của các lần ghi.

- **Điều kiện khẳng định ($\phi_{\text{assertion}}$):** Công thức này biểu diễn thuộc tính hoặc điều kiện cần được kiểm chứng trong chương trình. Nó thường là một khẳng định hoặc một điều kiện sau mà chương trình dự kiến sẽ đáp ứng.
- **Ràng buộc (ξ):** Trong ngữ cảnh của các chương trình đa luồng, thường cần có các ràng buộc bổ sung để mô hình hóa đồng bộ hóa và giao tiếp giữa các luồng. Các ràng buộc này có thể bao gồm thông tin về khóa, các hoạt động nguyên tử (atomic operations) hoặc các cơ chế đồng bộ hóa khác.

Bằng cách kết hợp các thành phần này, sự trừu tượng hóa của chương trình đa luồng có thể được biểu diễn dưới dạng một công thức logic được mô tả trong Công thức 3.1. Công thức này sau đó có thể được nhập vào bộ giải SMT để kiểm tra tính thỏa mãn của nó. Nếu công thức không thỏa mãn, điều đó có nghĩa là điều kiện khẳng định đúng trong tất cả các lần thực thi có thể có của chương trình. Nếu công thức có thể thỏa mãn, điều đó có nghĩa là tồn tại một lần thực thi chương trình vi phạm khẳng định và bộ giải SMT có thể cung cấp một ví dụ phản chứng để giúp xác định lỗi. Áp dụng công thức này vào chương trình đa luồng trên Hình 3.4, ta có công thức trừu tượng của chương trình như sau:

- $\phi_{\text{init}} = (i_{r01} = 3 \wedge j_{r01} = 6)$.
- $\rho = (\rho_{\text{thr1}} \wedge \rho_{\text{thr2}}) = (i_{w11} = j_{r11} + 1) \wedge (i_{w12} = j_{r12} + 1) \wedge (i_{w13} = j_{r13} + 1) \wedge (i_{w14} = j_{r14} + 1) \wedge (i_{w15} = j_{r15} + 1) \wedge (j_{w21} = i_{r21} + 1) \wedge (j_{w22} = i_{r22} + 1) \wedge (j_{w23} = i_{r23} + 1) \wedge (j_{w24} = i_{r24} + 1) \wedge (j_{w25} = i_{r25} + 1)$.
- ζ biểu diễn quy tắc: "Mỗi lần đọc của biến v có thể đọc kết quả của bất kỳ lần ghi nào của v ".
- $\phi_{\text{assertion}} = (i_{r31} > 16 \vee j_{r31} > 16)$.
- ξ = Các ràng buộc giải quyết đồng bộ hóa và giao tiếp giữa các luồng.

Mối quan hệ đọc - ghi (ζ) là một thành phần quan trọng trong thực thi tương trưng của các chương trình đa luồng. Nó nắm bắt mối quan hệ giữa các sự kiện đọc và ghi trên các biến chia sẻ, cho phép phân tích các xung đột dữ liệu tiềm ẩn và các vấn đề về đồng thời khác. Nguyên tắc cơ bản của ζ là "mỗi lần đọc của biến v có thể đọc kết quả của bất kỳ lần ghi nào của v ." Nguyên tắc này công nhận tính không xác định của việc thực thi đa luồng, nơi thứ tự thực hiện qua các luồng không luôn dự đoán được. Khoá luận này đưa ra hai quy tắc xây dựng các biểu diễn cho mối quan hệ đọc và ghi trên các biến chia sẻ trong chương trình đa luồng.

Quy tắc 1: Các biến đọc có thể được gán giá trị bởi bất kỳ biến ghi nào trong mã nguồn, bao gồm cả biến toàn cục. Do đó, công thức cho việc gán giá trị có thể của các biến đọc được tạo ra như sau:

$$\zeta = \bigwedge_{a=1, b=0}^k \bigvee_{m, n} (X_{ram} = X_{wbn}) \quad (3.2)$$

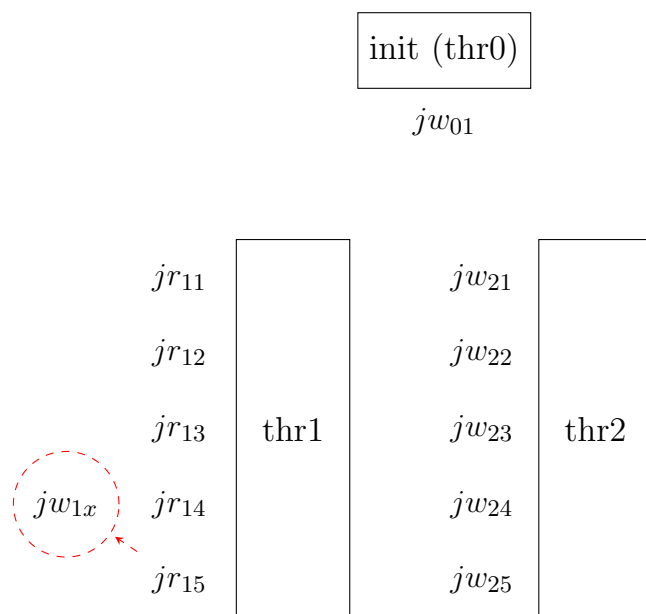
Nếu $a = b$ thì n là chỉ mục của biến ghi mới nhất mà đứng trước biến đọc trong cùng một luồng. Nói cách khác, các biến đọc có thể được gán bởi một biến ghi ngay trước nó trong cùng một luồng. Giả sử tại **thr1** có một biến ghi j_{w1x} giữa j_{r14} và j_{r15} , công thức sẽ bao gồm ($j_{r15} = j_{w1x}$) và được mô tả trong Hình 3.4. Giá trị của j_{r15} có thể được gán bởi j_{w01} hoặc j_{w11} hoặc j_{w12} hoặc j_{w13} hoặc j_{w14} hoặc j_{w15} .

Nếu $a \neq b$ thì n có thể là bất kỳ chỉ mục của biến ghi trong luồng khác. Nói cách khác, các biến đọc cũng có thể được gán bởi một biến ghi trong một luồng khác bất kỳ. Ví dụ như trên Hình 3.5, biến j_{r15} trong **thr1** có thể được gán giá trị bởi j_{w01} trong **thr0** hoặc các biến ghi j_{w21} , j_{w22} , j_{w23} , j_{w24} , j_{w25} trong **thr2**.

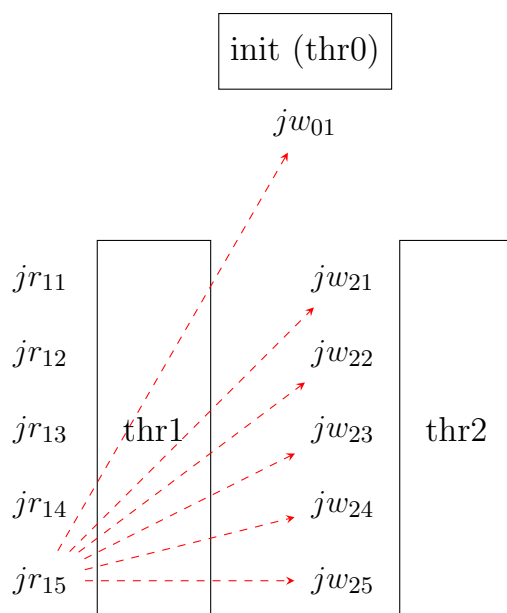
Cuối cùng, khi lấy ví dụ cho biến j_{r15} trong luồng 1 (**thr1**) và dựa trên mối quan hệ đọc - ghi (ζ), các giá trị có thể được gán cho j_{r15} là:

$$(j_{r15} = j_{w01}) \vee (j_{r15} = j_{w11}) \vee (j_{r15} = j_{w12}) \vee (j_{r15} = j_{w13}) \vee (j_{r15} = j_{w14}) \vee (j_{r15} = j_{w15})$$

Thuật toán 5 mô tả quá trình trừu tượng hóa chương trình đầu vào từ một đồ thị luồng điều khiển. Đầu vào của thuật toán là nút bắt đầu và nút kết thúc của chương trình, đầu ra là công thức trừu tượng của chương trình. Trong quá trình trừu tượng hóa, thuật toán duyệt qua các nút trong đồ thị luồng điều khiển, trích xuất công thức tương ứng của mỗi nút và kết hợp chúng lại với nhau. Quá



Hình 3.4: Các biến đọc có thể được gán bởi một biến ghi ngay trước nó trong cùng một luồng



Hình 3.5: Các biến đọc cũng có thể được gán bởi một biến ghi trong một luồng khác bất kỳ

Thuật toán 5: Trừu tượng hóa chương trình đầu vào

Đầu vào: start - nút bắt đầu của chương trình, exit - nút kết thúc của chương trình

Đầu ra: constraint - công thức trừu tượng của chương trình

```
1 Function create(start, exit):
2   constraint  $\leftarrow$  start.getFormula();
3   node  $\leftarrow$  start.getNext();
4   while node  $\neq$  null do
5     temp  $\leftarrow$  node.getFormula();
6     if temp  $\neq$  null then
7       if constraint = null then
8         | constraint  $\leftarrow$  temp;
9       end
10      else
11        | constraint  $\leftarrow$  wrapPrefix(LOGIC_AND, constraint, temp);
12      end
13    end
14    if node = exit then
15      | break;
16    end
17    if node is DecisionNode then
18      | node  $\leftarrow$  node.getEndNode();
19    end
20    else
21      | node  $\leftarrow$  node.getNext();
22    end
23  end
24  return constraint;
```

trình này tiếp tục cho đến khi đến nút kết thúc của chương trình hoặc không còn nút nào để duyệt. Kết quả cuối cùng là một công thức trừu tượng biểu diễn hành vi của chương trình. Mô hình trừu tượng được tạo thành từ công thức trừu tượng sau đó được sử dụng để kiểm chứng các thuộc tính của chương trình. Nhờ vào việc đơn giản hóa, quá trình kiểm chứng trở nên nhanh chóng và hiệu quả hơn, nhưng vẫn đảm bảo tính chính xác và đầy đủ.

Thuật toán 6: Biểu diễn mối quan hệ đọc - ghi ζ

Đầu vào: vm - quản lý biến, $operand$ - toán tử logic

Đầu ra: $constraint$ - các công thức logic biểu diễn mối quan hệ đọc - ghi

```

1 Function create( $vm$ ,  $operand$ ):
2    $inputList \leftarrow vm.getInputList();$ 
3   for  $readVar$  in  $inputList$  do
4      $varName \leftarrow readVar.getName();$ 
5      $writeVarList \leftarrow getWriteVarList(vm, varName);$ 
6      $constraintTemp2 \leftarrow "";$ 
7     for  $writeVar$  in  $writeVarList$  do
8        $constraintTemp3 \leftarrow wrapPrefix(operand,$ 
9          $readVar.getVariableWithIndex(),$ 
10         $writeVar.getVariableWithIndex());$ 
11        $constraintTemp2 \leftarrow wrapPrefix(LOGIC\_OR, constraintTemp2,$ 
12         $constraintTemp3);$ 
13     end
14    $constraint.add(constraintTemp2);$ 
15 end
16 return  $constraint;$ 

```

Thuật toán 6 mô tả cách biểu diễn mối quan hệ đọc - ghi ζ trong chương trình đa luồng. Đầu vào của thuật toán là quản lý biến vm và toán tử logic $operand$, đầu ra là danh sách các công thức logic biểu diễn mối quan hệ đọc - ghi. Trong quá trình thực thi, thuật toán duyệt qua danh sách các biến đọc trong chương trình, tìm kiếm danh sách các biến ghi tương ứng và tạo ra các công thức logic biểu diễn mối quan hệ giữa chúng. Kết quả cuối cùng là một danh sách các công thức logic biểu diễn mối quan hệ đọc - ghi giữa các biến trong chương trình. Các công thức

này sau đó được sử dụng để xác định các giá trị có thể được gán cho các biến đọc trong chương trình.

3.2.5 Pha xây dựng công thức cho ràng buộc giải quyết xung đột đọc - ghi

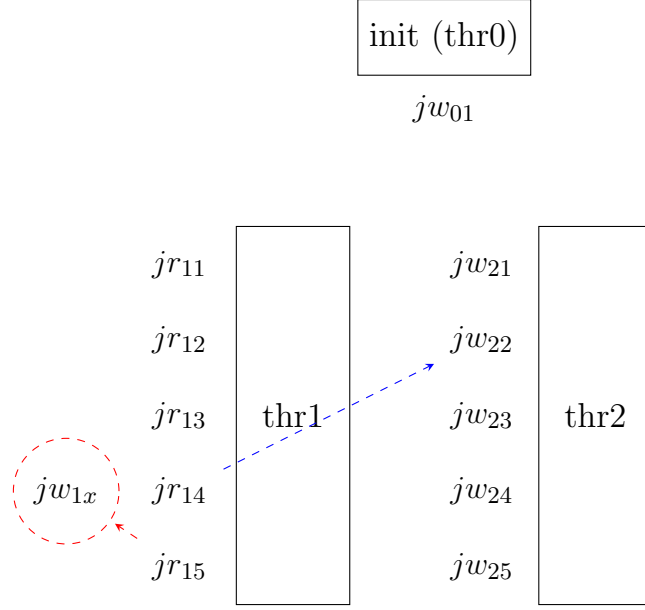
Pha xây dựng công thức cho ràng buộc giải quyết xung đột đọc - ghi là đề xuất chính của phương pháp kiểm chứng chương trình đa luồng dựa trên thực thi tượng trưng, nhằm đảm bảo tính nhất quán và tránh các lỗi xảy ra do truy cập đồng thời đến các biến chia sẻ. Trước tiên, tất cả các biến chia sẻ giữa các luồng trong chương trình được xác định. Đây là những biến có thể bị truy cập bởi nhiều luồng khác nhau, dẫn đến khả năng xảy ra xung đột đọc-ghi. Với mỗi biến chia sẻ, các điểm truy cập (đọc và ghi) trong các luồng khác nhau được phân tích. Điều này bao gồm việc xác định thứ tự và ngữ cảnh của mỗi lần truy cập biến trong từng luồng. Các truy cập đọc và ghi của mỗi biến được đánh chỉ mục để xác định thứ tự và vị trí chính xác của chúng trong luồng thực thi. Chỉ mục này giúp theo dõi chính xác thứ tự truy cập và phát hiện các xung đột tiềm ẩn. Dựa trên phân tích truy cập biến, các công thức ràng buộc được xây dựng để đảm bảo không có xung đột đọc-ghi xảy ra. Dưới đây là quy tắc về việc xây dựng công thức cho ràng buộc giải quyết xung đột đọc-ghi.

Quy tắc 2: Khi một biến đọc r được gán giá trị bởi một biến ghi w (không cùng luồng), biến đọc ngay sau r (nếu có) sẽ được gán giá trị bởi một trong ba trường hợp.

$$\xi = \bigwedge_{a=1, b=0}^k \left((X_{ram} = X_{wbn}) \Rightarrow \left[\bigvee_{a \neq b}^k (X_{rau} = X_{wkv}) \right] \right) \quad (3.3)$$

Nếu $k = a$, v là chỉ mục của biến ghi gần đây nhất. Hay nói cách khác, biến đọc sẽ được gán giá trị bởi biến ghi gần đây nhất. Trên Hình 3.6, biến đọc j_{r15} sẽ được gán giá trị bởi j_{w1x} nếu tồn tại trường hợp mà biến j_{w1x} tồn tại giữa j_{r14} và j_{r15} , đồng thời biến đọc j_{r14} được gán giá trị bởi j_{w22} .

Nếu $b = k$, $v \geq n$. Nói cách khác, biến đọc thứ hai sẽ được gán giá trị bởi các biến ghi có chỉ mục thứ tự lớn hơn hoặc bằng của chính nó, xét trên luồng của biến đang ghi cho biến đọc thứ nhất. Trong trường hợp trên Hình 3.7, biến đọc j_{r15} sẽ



Hình 3.6: Biểu diễn ràng buộc giải quyết xung đột đọc - ghi 1

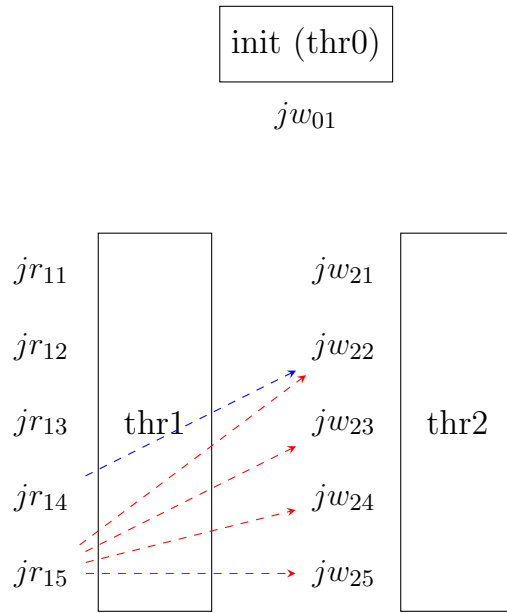
được gán giá trị bởi một trong các biến ghi j_{w22} , j_{w23} , j_{w24} hoặc j_{w25} khi biến đọc j_{r14} được gán giá trị bởi j_{w22} .

Nếu $k \neq a$, v có thể là bất kỳ giá trị nào. Nói cách khác, biến đọc sẽ được gán giá trị bởi một trong các biến ghi khác nhau tại các luồng khác với các luồng đang xét. Trong trường hợp trên Hình 3.8, biến đọc j_{r15} sẽ được gán giá trị bởi một trong các biến ghi j_{wNx} hoặc j_{wNy} , giả sử tồn tại trường hợp mà luồng thứ N chứa hai biến j_{wNx} và j_{wNy} , đồng thời biến đọc j_{r14} được gán giá trị bởi j_{w22} .

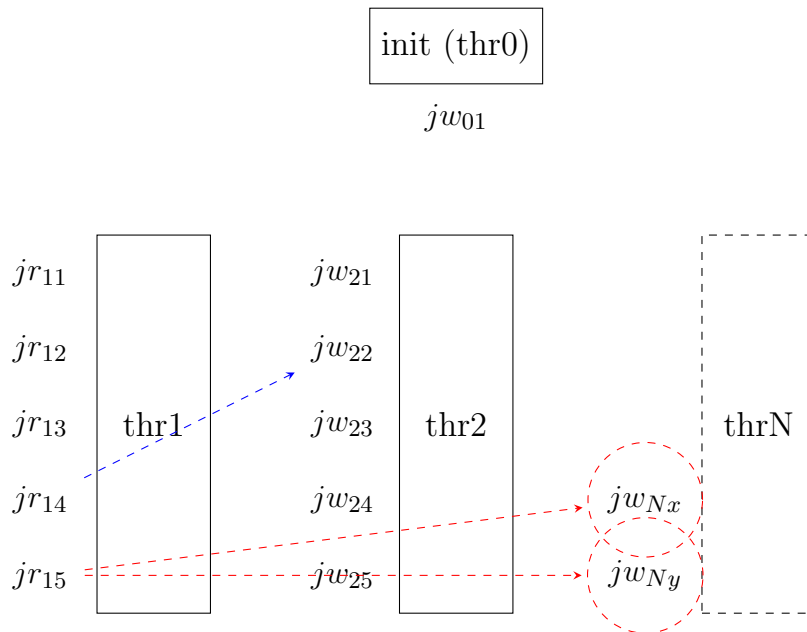
Cuối cùng, khi lấy ví dụ về việc đọc biến j_{r14} từ giá trị của biến j_{w22} , công thức ràng buộc sẽ được biểu diễn như sau:

$$\begin{aligned}
 &(j_{r14} = j_{w22}) \Rightarrow \\
 &[(j_{r15} = j_{w1x})] \vee \\
 &[(j_{r15} = j_{w22}) \vee (j_{r15} = j_{w23}) \vee (j_{r15} = j_{w24}) \vee (j_{r15} = j_{w25})] \vee \\
 &[(j_{r15} = j_{wNx}) \vee (j_{r15} = j_{wNy})]
 \end{aligned}$$

Thuật toán 7 mô tả quá trình xây dựng các công thức ràng buộc giải quyết xung đột đọc - ghi. Đầu vào của thuật toán là một đối tượng quản lý biến vm và đầu ra là danh sách các công thức ràng buộc. Đầu tiên, thuật toán lấy danh sách các biến đầu vào $inputList$ từ đối tượng quản lý biến. Sau đó, với mỗi biến đọc $readVar$ trong danh sách $inputList$, thuật toán lấy danh sách các biến ghi $writeListSameThread$



Hình 3.7: Biểu diễn ràng buộc giải quyết xung đột đọc - ghi 2



Hình 3.8: Biểu diễn ràng buộc giải quyết xung đột đọc - ghi 3

Thuật toán 7: Xây dựng công thức giải quyết xung đột đọc - ghi

Đầu vào: *vm* - Quản lý biến

Đầu ra: *constraints* - các công thức giải quyết xung đột đọc - ghi

```
1 Function buildConstraints(vm):
2   inputList  $\leftarrow$  vm.getInputList();
3   for each readVar in inputList do
4     writeListSameThread  $\leftarrow$  getWriteListInSameThread(vm, readVar);
5     for each writeVar in writeListSameThread do
6       | initConstraint  $\leftarrow$  wrapPrefix(operand, readVar, writeVar);
7     end
8     for each writeVar in inputList do
9       | initConstraint  $\leftarrow$  wrapPrefix(LOGIC_OR, initConstraint,
10      | wrapPrefix(operand, readVar, writeVar));
11    end
12    constraints.add(initConstraint);
13    writeListOtherThread  $\leftarrow$  getWriteListInOtherThread(vm, readVar);
14    for each writeVar in writeListSameThread do
15      | constraintTemp  $\leftarrow$  "";
16      for each readVar2 in readListInSameThread(vm, readVar) do
17        | for each writeVar2 in writeListInSameThread(vm, readVar2) do
18          | constraintTemp  $\leftarrow$  wrapPrefix(...);
19        | end
20        for each writeVar2 in inputList do
21          | constraintTemp  $\leftarrow$  wrapPrefix(...);
22        | end
23      | end
24      constraints.add(wrapPrefix(BINARY_CONNECTIVE,
25      | wrapPrefix(...)));
26    end
27  end
28  return constraints;
```

trong cùng một luồng với biến đọc. Một ràng buộc khởi tạo *initConstraint* được tạo ra bằng cách gói biến đọc và biến ghi vào một hàm *wrapPrefix*. Tiếp theo, với mỗi biến ghi *writeVar* trong danh sách *writeListSameThread*, ràng buộc khởi tạo được cập nhật bằng cách gói biến đọc và biến ghi vào một hàm *wrapPrefix* khác. Cuối cùng, ràng buộc khởi tạo được thêm vào danh sách ràng buộc *constraints*. Quá trình tạo ràng buộc cho các biến ghi trong các luồng khác nhau được thực hiện tương tự như trên.

3.2.6 Pha thực thi tượng trưng

Để kiểm chứng được mã nguồn chương trình đầu vào, một tệp XML chứa các điều kiện khẳng định và các tính chất an toàn cần được tạo ra. Đây thường là tệp mà người dùng cung cấp, chứa các điều kiện mà họ muốn chương trình thỏa mãn. Pha thực thi tượng trưng sẽ đọc từ tệp XML này để xây dựng đầy đủ công thức biểu diễn toàn bộ trạng thái và các ràng buộc của chương trình dưới dạng công thức logic vị từ. Công thức này sẽ bao gồm các ràng buộc của chương trình, các điều kiện khẳng định và các điều kiện an toàn. Công thức này sẽ được sử dụng trong pha thực thi tượng trưng để kiểm tra xem chương trình có thỏa mãn các điều kiện đã đặt ra hay không.

Thuật toán 8 mô tả quá trình đọc thông tin từ tệp XML chứa các điều kiện khẳng định từ người dùng. Đầu vào của thuật toán là tệp XML chứa thông tin, đầu ra là danh sách các điều kiện khẳng định. Trong quá trình đọc, thuật toán trích xuất các phần tử *AssertionMethod* từ tệp XML và tạo một danh sách các đối tượng *AssertionMethod* tương ứng. Mỗi *AssertionMethod* bao gồm thông tin về phương thức, điều kiện trước và điều kiện sau. Danh sách này sẽ được sử dụng trong pha thực thi tượng trưng để xây dựng công thức biểu diễn toàn bộ trạng thái và các ràng buộc của chương trình.

Sau khi đã xây dựng được công thức biểu diễn toàn bộ trạng thái và các ràng buộc của chương trình dưới dạng công thức logic vị từ, pha này sẽ sử dụng công thức đó làm đầu vào cho một bộ giải SMT, chẳng hạn như Z3. Bộ giải này sẽ cố gắng xác định xem công thức có thỏa mãn được hay không. Nếu bộ giải SMT xác định rằng công thức không thỏa mãn, điều này có nghĩa là chương trình thỏa mãn các tính chất an toàn và điều kiện khẳng định của người dùng dưới tất cả các cách xen kẽ có thể của các luồng. Nếu công thức có thể thỏa mãn, điều này cho thấy

Thuật toán 8: Đọc thông tin từ tệp XML

Đầu vào: file - tệp XML chứa thông tin từ người dùng

Đầu ra: assertions - danh sách các điều kiện khẳng định

1 Function readXML(*file*):

```
2   factory ← DocumentBuilderFactory.newInstance();
3   builder ← factory.newDocumentBuilder();
4   doc ← loadXMLFromFile(file);
5   nodeList ← doc.getElementsByTagName("AssertionMethod");
6   for i ← 0 to nodeList.getLength() - 1 do
7       node ← nodeList.item(i);
8       nodeListTemp ← node.getChildNodes();
9       method ← null;
10      preCondition ← null;
11      postCondition ← null;
12      for j ← 0 to nodeListTemp.getLength() - 1 do
13          n ← nodeListTemp.item(j);
14          if n.getNodeName().equals("Method") then
15              | method ← n.getTextContent();
16          end
17          else if n.getNodeName().equals("PreCondition") then
18              | preCondition ← n.getTextContent();
19          end
20          else if n.getNodeName().equals("PostCondition") then
21              | postCondition ← n.getTextContent();
22          end
23      end
24      list.add(new AssertionMethod(method, preCondition,
25                                   postCondition));
26  end
27  return list;
```

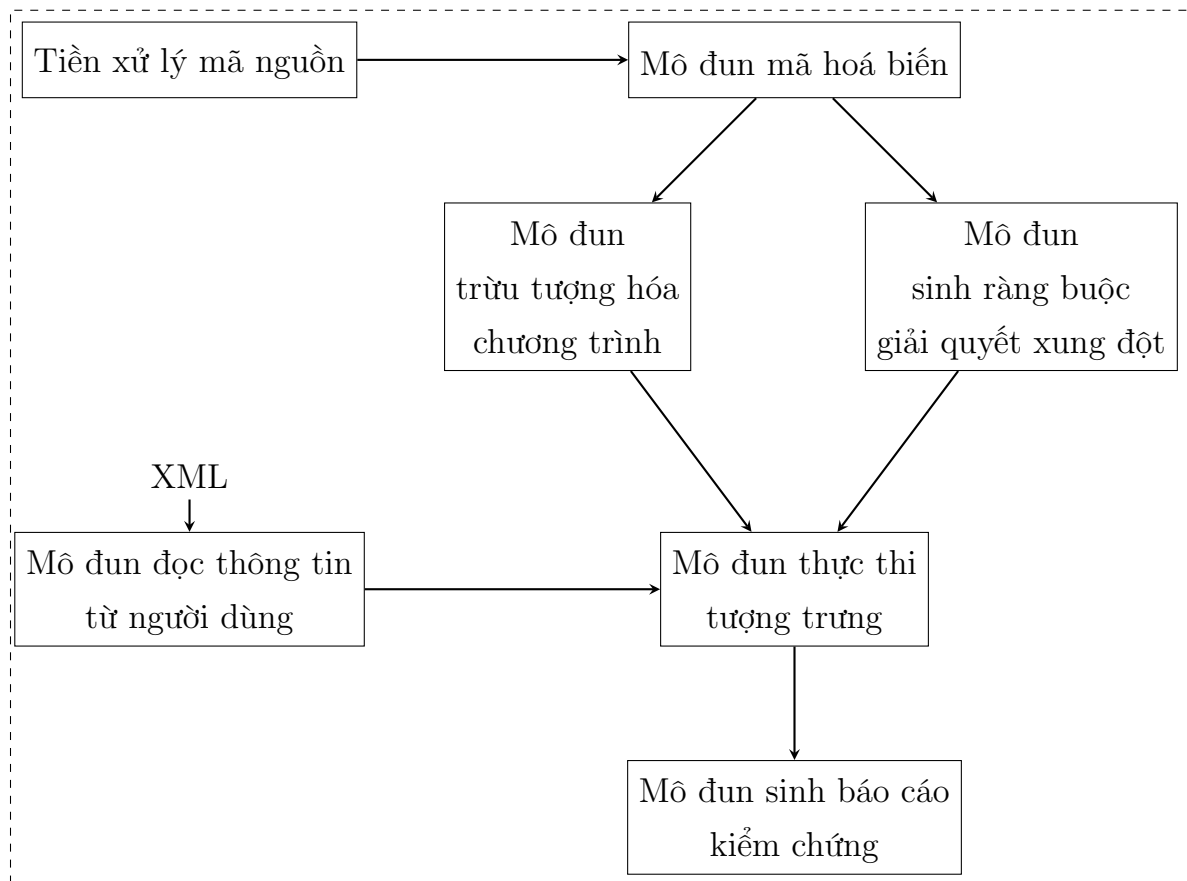
rằng tồn tại một trường hợp phản chứng mà các tính chất an toàn hoặc điều kiện khẳng định bị vi phạm. Trong trường hợp công thức có thể thỏa mãn, công cụ sẽ tạo ra một báo cáo kiểm chứng nêu bật các lỗi tiềm ẩn. Báo cáo này cung cấp thông tin chi tiết về trường hợp phản chứng, giúp người dùng xác định và sửa lỗi trong mã nguồn. Pha thực thi tượng trưng giúp đảm bảo chương trình hoạt động đúng đắn và tuân thủ các điều kiện đã đặt ra bằng cách kiểm tra mọi khả năng có thể xảy ra trong quá trình thực thi đa luồng. Việc sử dụng bộ giải SMT giúp tự động hóa quá trình kiểm chứng và cung cấp các kết quả chính xác, giúp phát hiện và sửa lỗi hiệu quả.

Chương 4

Cài đặt công cụ và Thực nghiệm

4.1 Cài đặt công cụ

Để tiến hành thực nghiệm cho phương pháp đề xuất, Khoa luận đã cài đặt một công cụ kiểm chứng chương trình đa luồng dựa trên kỹ thuật thực thi tượng trưng có tên là MVS (Multithreaded program Verification tool based on Symbolic execution). Công cụ này được xây dựng trên nền tảng Java và sử dụng thư viện Z3 để giải quyết các ràng buộc logic. MVS hỗ trợ kiểm chứng chương trình đa luồng viết bằng ngôn ngữ Java.



Hình 4.1: Kiến trúc tổng quan của công cụ

Hình 4.1 mô tả kiến trúc tổng quan của công cụ MVS. Trước tiên, mã nguồn

sẽ được tiền xử lý để loại bỏ các phần không cần thiết như comment, các khai báo không cần thiết, v.v. Sau đó, mã nguồn sẽ được đưa vào mô đun mã hoá biến để xác định các biến đọc và ghi. Sau đó, mã nguồn sẽ được truyền vào mô đun trừu tượng hóa chương trình để xây dựng cây cú pháp trừu tượng và đồ thị luồng điều khiển. Mô đun sinh ràng buộc giải quyết xung đột sẽ sinh ra các ràng buộc giải quyết xung đột đọc-ghi từ cây cú pháp trừu tượng và đồ thị luồng điều khiển. Mô đun thực thi tượng trưng sẽ sử dụng các ràng buộc giải quyết xung đột để thực thi tượng trưng chương trình. Mô đun đọc thông tin từ người dùng sẽ đọc thông tin từ tệp XML chứa các điều kiện khẳng định và truyền vào mô đun thực thi tượng trưng. Cuối cùng, mô đun sinh báo cáo kiểm chứng sẽ sinh ra báo cáo kiểm chứng dựa trên kết quả thực thi tượng trưng.

4.1.1 Tiền xử lý mã nguồn

Pha tiền xử lý mã nguồn là bước để chuẩn bị mã nguồn cho quá trình phân tích chương trình. Trong khoá luận này, mã nguồn là các chương trình đa luồng viết bằng ngôn ngữ C/C++, cần được định dạng theo tiêu chuẩn cụ thể để phù hợp với phân tích chương trình. Dưới đây là một ví dụ về mã nguồn đầu vào và mã nguồn sau khi tiền xử lý.

Quan sát từ hai Hình 4.2 và 4.3, ta thấy rằng mã nguồn sau khi tiền xử lý đã loại bỏ các phần không cần thiết như phần khai báo thư viện, các hằng số và các biểu thức tính toán liên quan tới hằng số. Các biến `NUM` và `LIMIT` đã được định nghĩa trước khi sử dụng. Các lệnh liên quan đến lập trình đa luồng và kiểm chứng điều kiện như các hàm `pthread_create`, `pthread_exit`, `__VERIFIER_atomic_begin`, `__VERIFIER_atomic_end`, `reach_error` và `abort` cũng được thay thế bằng các dạng đơn giản hơn, làm cho quá trình phân tích trở nên dễ dàng hơn. Ngoài ra, các biến `i` và `j` đã được khai báo ở đầu chương trình và các hàm `t1` và `t2` đã được tách ra từ hàm `main`. Cuối cùng, một hàm `triangular_1` đã được thêm vào để thực thi chương trình dựa theo tên của chương trình được lấy từ tên tệp mã nguồn. Tại thời điểm viết khoá luận này, công cụ chưa xử lý được các trường hợp mã nguồn chứa vòng lặp hoặc các lệnh nhảy như `goto`. Do đó, công cụ sẽ loại bỏ các lệnh nhảy và các vòng lặp trong mã nguồn được chuyển đổi thành các câu lệnh lặp lại. Ví dụ, vòng lặp `for` trong mã nguồn chưa được tiền xử lý sẽ được chuyển đổi thành các câu lệnh lặp lại như trong mã nguồn sau khi tiền xử lý.

```

#include <pthread.h>
extern void __VERIFIER_atomic_begin();
extern void __VERIFIER_atomic_end();
extern void abort(void);
#include <assert.h>
void reach_error() {
    assert(0);
}
int i = 3, j = 6;
#define NUM 5
#define LIMIT (2 * NUM + 6)
void *t1(void *arg) {
    for (int k = 0; k < NUM; k++) {
        __VERIFIER_atomic_begin();
        i = j + 1;
        __VERIFIER_atomic_end();
    }
    pthread_exit(NULL);
}
void *t2(void *arg) {
    ...
}
int main(int argc, char *argv[]) {
    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, t1, NULL);
    pthread_create(&thread2, NULL, t2, NULL);
    __VERIFIER_atomic_begin();
    int condI = i > LIMIT;
    __VERIFIER_atomic_end();
    ...
    if (condI || condJ) {
        ERROR: {reach_error(); abort();}
    }
    return 0;
}

```

Hình 4.2: Ví dụ về mã nguồn trước khi tiến xử lý

```

int i = 3;
int j = 6;
void *t1() {
    i = j + 1;
    i = j + 1;
    i = j + 1;
    i = j + 1;
    i = j + 1;
}
void *t2() {
    j = i + 1;
    j = i + 1;
    j = i + 1;
    j = i + 1;
    j = i + 1;
}
int triangular_1() {
    t1();
    t2();
    if (i > 16 || j > 16) {
        return 1;
    } else {
        return 0;
    }
}

```

Hình 4.3: Ví dụ về mã nguồn sau khi tiền xử lý

Hiện tại, việc chuyển đổi mã nguồn đầu vào đang được thực hiện thủ công mặc dù quá trình này khá đơn giản. Tuy nhiên, trong tương lai, công cụ có thể được cải tiến để thực hiện chuyển đổi tự động, giúp giảm thiểu công sức và thời gian của người sử dụng.

4.1.2 Mô đun mã hoá biến

Mô đun mã hoá biến có nhiệm vụ xác định các biến đọc và ghi trong mã nguồn đầu vào. Để thực hiện điều này, công cụ sử dụng thư viện Eclipse CDT để xây dựng cây cú pháp trừu tượng từ mã nguồn đầu vào (như đã mô tả trong phần 3.2.1). Sau khi đã xây dựng được cây cú pháp trừu tượng, công cụ sẽ duyệt cây cú pháp trừu tượng và xác định các biến đọc và ghi trong mã nguồn. Cụ thể, công cụ sẽ xác định các biến đọc và ghi trong các lệnh gán, lệnh đọc và lệnh ghi của mã nguồn cũng như các thành phần khác như điều kiện của các lệnh rẽ nhánh. Tiếp theo, công cụ sẽ xây dựng một đồ thị luồng điều khiển từ cây cú pháp trừu tượng để xác định các đường đi có thể thực thi trong chương trình. Khi có được các biến đọc và ghi cùng với đồ thị luồng điều khiển, công cụ sẽ tiến hành đánh chỉ mục các biến đọc và ghi trong mã nguồn.

4.1.3 Mô đun trừu tượng hóa chương trình

Mô đun trừu tượng hóa chương trình có nhiệm vụ xây dựng toàn bộ biểu diễn của chương trình đầu vào và các giá trị có thể được gán cho các biến đọc (tham khảo phần 3.2.4). Để thực hiện điều này, công cụ sẽ sử dụng cây cú pháp trừu tượng và đồ thị luồng điều khiển đã được xây dựng từ mô đun mã hoá biến, từ đây thông tin về các biến, hàm, và cấu trúc điều khiển trong chương trình được trích xuất. Các khối mã lệnh phức tạp được thay thế bằng các biểu diễn trừu tượng đơn giản hơn. Ví dụ, các vòng lặp và cấu trúc điều khiển phức tạp có thể được thay thế bằng các khối mã lệnh tương đương nhưng đơn giản hơn. Từ thông tin đã trích xuất, một biểu diễn trừu tượng của chương trình được tạo ra. Biểu diễn này bao gồm các trạng thái của chương trình và các biến đổi giữa các trạng thái này. Các trạng thái và biến đổi được biểu diễn dưới dạng các công thức logic vị từ, giúp dễ dàng áp dụng các kỹ thuật phân tích hình thức. Biểu diễn trừu tượng của chương trình sau đó kết hợp với biểu diễn cho ràng buộc giải quyết xung đột đọc-ghi để

tạo ra một biểu diễn hoàn chỉnh của chương trình và đưa vào mô đun thực thi tượng trưng.

4.1.4 Mô đun sinh ràng buộc giải quyết xung đột

Mô đun sinh ràng buộc giải quyết xung đột có nhiệm vụ tạo ra các ràng buộc logic để xác định và xử lý các xung đột có thể xảy ra giữa các biến đọc và viết trong các luồng khác nhau. Để thực hiện điều này, công cụ sẽ sử dụng thông tin về các biến đọc và ghi đã được đánh chỉ mục từ mô đun mã hoá biến và biểu diễn trừu tượng của chương trình từ mô đun trừu tượng hóa chương trình. Các ràng buộc giải quyết xung đột đọc-ghi sẽ được sinh ra từ các biến đọc và ghi trong chương trình. Cụ thể, công cụ sẽ xác định các biến đọc và ghi trong các lệnh gán, lệnh đọc và lệnh ghi của mã nguồn cũng như các thành phần khác như điều kiện của các lệnh rẽ nhánh. Sau đó, dựa trên các quy tắc đã được xác định trong phần 3.2.5, công cụ sẽ sinh ra các ràng buộc giải quyết xung đột đọc-ghi từ các biến đọc và ghi đã xác định. Các ràng buộc này kết hợp với biểu diễn trừu tượng của chương trình để tạo ra một biểu diễn hoàn chỉnh của chương trình và đưa vào mô đun thực thi tượng trưng.

4.1.5 Mô đun đọc thông tin từ người dùng

Như đã mô tả trong phần 3.2.6, mô đun này sẽ đọc thông tin từ tệp XML chứa các điều kiện khẳng định từ người dùng. Dưới đây là một ví dụ về tệp XML chứa thông tin về điều kiện khẳng định (Hình 4.4). Trong ví dụ này, tệp XML chứa một điều kiện khẳng định cho hàm `triangular_1` với điều kiện rằng hàm sẽ trả về giá trị 0. Các điều kiện khẳng định này sẽ được sử dụng trong pha thực thi tượng trưng để xây dựng công thức biểu diễn toàn bộ trạng thái và các ràng buộc của chương trình.

triangular_1.xml
<pre> <AssertionFile> <AssertionMethod> <Method>triangular_1</Method> <PreCondition></PreCondition> <PostCondition>return = 0</PostCondition> </AssertionMethod> </AssertionFile> </pre>

Hình 4.4: Ví dụ về tệp XML chứa thông tin về điều kiện khẳng định

Để thực hiện điều này, công cụ sẽ sử dụng thư viện Java DOM để đọc thông tin từ tệp XML. Cụ thể, công cụ sẽ trích xuất các phần tử *AssertionMethod* từ tệp XML và tạo một danh sách các đối tượng *AssertionMethod* tương ứng. Mỗi *AssertionMethod* bao gồm thông tin về phương thức, điều kiện trước và điều kiện sau. Danh sách này sẽ được sử dụng trong pha thực thi tượng trưng để xây dựng công thức biểu diễn toàn bộ trạng thái và các ràng buộc của chương trình.

4.1.6 Mô đun thực thi tượng trưng

Mô đun thực thi tượng trưng có nhiệm vụ thực thi tượng trưng chương trình dựa trên biểu diễn trừu tượng của chương trình và các ràng buộc giải quyết xung đột đọc-ghi. Mô đun này thực hiện việc thực thi chương trình bằng cách sử dụng các biểu thức tượng trưng thay vì các giá trị cụ thể, giúp khám phá tất cả các khả năng có thể xảy ra trong quá trình chạy chương trình (đã được trình bày trong phần 3.2.6). Để thực hiện điều này, công cụ sẽ sử dụng bộ giải SMT Z3 để giải quyết các ràng buộc logic. Cụ thể, công cụ sẽ sử dụng công thức biểu diễn toàn bộ trạng thái và các ràng buộc của chương trình dưới dạng công thức logic vị từ làm đầu vào cho bộ giải SMT. Bộ giải này sẽ cố gắng xác định xem công thức có thỏa mãn được hay không. Nếu bộ giải SMT xác định rằng công thức không thỏa mãn, điều này có nghĩa là chương trình thỏa mãn các tính chất an toàn và điều kiện khẳng định của người dùng dưới tất cả các cách xen kẽ có thể của các luồng. Nếu công thức có thể thỏa mãn, điều này cho thấy rằng tồn tại một trường hợp phản chứng mà các tính chất an toàn hoặc điều kiện khẳng định bị vi phạm. Trong trường hợp công thức có thể thỏa mãn, công cụ sẽ tạo ra một báo cáo kiểm chứng

nêu bật các lỗi tiềm ẩn, giúp người phát triển chương trình xác định và khắc phục lỗi.

4.1.7 Mô đun sinh báo cáo kiểm chứng

Mô đun sinh báo cáo kiểm chứng có nhiệm vụ sinh báo cáo kiểm chứng dựa trên kết quả thực thi tượng trưng. Báo cáo này cung cấp thông tin chi tiết về trường hợp phản chứng, giúp người dùng xác định và sửa lỗi trong mã nguồn. Nếu công thức không thỏa mãn, ghi nhận rằng chương trình thỏa mãn các điều kiện an toàn. Nếu công thức thỏa mãn, phân tích các phản chứng để xác định các tình huống vi phạm tính an toàn của chương trình. Báo cáo kiểm chứng cũng cung cấp thông tin về các điều kiện khẳng định đã được kiểm tra và kết quả của việc kiểm tra. Báo cáo này giúp người dùng hiểu rõ hơn về các lỗi tiềm ẩn trong mã nguồn và cách sửa chúng.

4.2 Kết quả thực nghiệm

Để đánh giá hiệu quả của phương pháp đề xuất, khoá luận này đã thực hiện một loạt các thực nghiệm trên các chương trình đa luồng tiêu biểu.

4.2.1 Môi trường thực nghiệm

Các thực nghiệm đã được thực hiện trên một máy tính cá nhân với cấu hình AMD Ryzen 7 3700U CPU và 20GB RAM. Máy tính chạy hệ điều hành Windows 11 và sử dụng công cụ giải SMT Z3 phiên bản mới nhất.

Trong quá trình thực hiện luận văn, chương trình đã được kiểm tra trên các tập dữ liệu *pthread*, *pthread-nondet*, *pthread-deagle*, và *pthread-atomic* của cuộc thi SV-COMP, tất cả đều được viết bằng ngôn ngữ C. SV-COMP là một cuộc thi uy tín diễn ra hàng năm, tập trung vào việc đánh giá các công cụ kiểm chứng phần mềm nhằm xác định công cụ có khả năng giải quyết nhiều vấn đề nhất trong thời gian ngắn nhất. Mặc dù SV-COMP có nhiều loại bài toán khác nhau, báo cáo luận văn này chỉ sử dụng ba tập dữ liệu trên làm tập dữ liệu thử nghiệm cho công cụ MVS do giới hạn về khả năng của công cụ. Trong phạm vi luận văn này, một số bài toán trong tập dữ liệu *pthread-nondet* đã được điều chỉnh lại để giống với các

triangular-1.yml
<pre> format_version: '2.0' input_files: 'triangular-1.i' properties: - property_file: ../properties/unreach-call.prp expected_verdict: true - property_file: ../properties/valid-memsafety.prp expected_verdict: true - property_file: ../properties/no-overflow.prp expected_verdict: true - property_file: ../properties/no-data-race.prp expected_verdict: true options: language: C data_model: ILP32 </pre>

Hình 4.5: Một tập YAML chứa thông tin về kết quả kì vọng của bài toán *triangular-1*

bài toán trong tập dữ liệu pthread nhằm giảm độ phức tạp của chương trình đầu vào cho công cụ MVS.

Mỗi bài toán bao gồm mã nguồn và các điều kiện cần kiểm tra do người dùng cung cấp, theo định dạng đã mô tả trong phần trước của luận văn. Kết quả đầu ra của mỗi bài toán có thể thuộc một trong ba trường hợp. Nếu công cụ MVS xác định rằng điều kiện của người dùng được thỏa mãn, kết quả được coi là *True*. Nếu công cụ MVS xác định rằng điều kiện của người dùng bị vi phạm và tạo ra một trường hợp phản chứng, kết quả được coi là *False*. Nếu công cụ MVS không thể đưa ra kết quả trong thời gian quy định, kết quả được coi là *Timeout*. Các kết quả kì vọng của các bài toán đã được xác định trước và được sử dụng để đánh giá hiệu suất của công cụ MVS.

Tập YAML trong Hình 4.5 chứa thông tin về kết quả kì vọng của bài toán *triangular-1*. Tập này bao gồm các thông tin về tập mã nguồn đầu vào, các điều kiện cần kiểm tra, và kết quả kì vọng của mỗi điều kiện. Các điều kiện cần kiểm tra bao gồm *unreach-call*, *valid-memsafety*, *no-overflow*, và *no-data-race*. Kết quả

Bảng 4.1: Kết quả thực nghiệm trên các tập dữ liệu *pthread*, *pthread-nondet*, *pthread-deagle*, và *pthread-atomic*

STT	Bài toán	Kết quả kì vọng	Kết quả	Số ràng buộc	Tổng thời gian(s)
1	stateful01_1 (1)	false	false	41	0.025
2	stateful01_2(1)	false	false	41	0.014
3	triangular_1 (1)	true	true	117	0.079
4	triangular_2(1)	false	false	117	0.098
5	triangular_longer_1 (1)	true	true	317	9.05
6	triangular_longer_2(1)	true	true	317	10.78
7	triangular_longest_1 (1)	true	/	1017	timeout
8	triangular_longest_1 (1)	true	/	1017	timeout
9	read_write_lock_1 (2)	true	true	111	0.144
10	read_write_lock_1b (2)	true	true	103	0.04
11	read_write_lock_2 (2)	true	true	127	0.031
12	peterson (2)	true	true	53	0.037
13	reorder_c11_bad_10 (3)	false	false	87	0.15
14	reorder_c11_bad_30 (3)	false	false	207	0.089
15	reorder_c11_good_10 (3)	false	false	87	0.045
16	reorder_c11_good_30 (3)	false	false	207	0.072
17	nondet_loop_bound_1 (4)	false	false	177	0.196
18	nondet_loop_bound_2 (4)	false	false	178	0.115

kì vọng của mỗi điều kiện có thể là *True*, *False*, hoặc *Timeout*.

4.2.2 Kết quả

Bảng 4.1 thể hiện kết quả thực nghiệm của công cụ MVS trên các tập dữ liệu *pthread*, *pthread-nondet*, *pthread-deagle*, và *pthread-atomic*. Các bài toán đã được chia thành các nhóm tương ứng với tập dữ liệu mà chúng thuộc về. Trong bảng kết quả, các giá trị *true* biểu thị rằng chuỗi thực thi vi phạm không xảy ra (UNSAT), trong khi các giá trị *false* cho thấy chuỗi thực thi vi phạm xảy ra và một phản

chứng được tạo ra (SAT). Cột *Số ràng buộc* thể hiện số lượng ràng buộc được tạo ra trong quá trình kiểm chứng. Cột *Tổng thời gian* thể hiện tổng thời gian mà công cụ MVS thực hiện kiểm chứng cho mỗi bài toán. Các bài toán từ 1 đến 8 được lấy từ tập thử nghiệm *pthread* (1), các bài toán từ 9 đến 12 từ tập thử nghiệm *pthread-atomic* (2), các bài toán từ 13 đến 16 từ tập thử nghiệm *pthread-deagle* (3), và các bài toán 17, 18 từ tập thử nghiệm *pthread-nondet* (4). Kết quả trả về đã được so sánh và đối chiếu với kết quả kì vọng của các bài toán tương ứng. Kết quả cho thấy công cụ kiểm chứng MVS đã chạy đúng hầu hết (16/18) các bài toán đầu vào.

Hình 4.6 thể hiện kết quả kiểm chứng của một bài toán. Kết quả kiểm chứng bao gồm trạng thái của bài toán (*true*), tổng thời gian kiểm chứng (410ms), và kết quả trả về từ bộ giải SMT Z3 (*unsat*). Các thông số khác như số lần giải mâu thuẫn, số lần quyết định, số lần giảm biến, và thời gian thực thi cũng được hiển thị trong kết quả.

Trong khoá luận này, do giới hạn về phần cứng, nên không thể thực hiện thêm nhiều thực nghiệm để đánh giá hiệu suất của công cụ MVS trên các tập dữ liệu lớn hơn cũng như thực nghiệm để so sánh với các công cụ kiểm chứng khác. Tuy nhiên từ góc độ phương pháp luận, có thể thấy rằng phương pháp đề xuất có ưu điểm về mặt triển khai vì chỉ cần được đưa vào bộ giải SMT một lần, thay vì nhiều lần như phương pháp "Scheduling Constraint Based Abstraction Refinement". Bên cạnh đó, phương pháp này cũng đơn giản hơn so với việc triển khai các lệnh thực thi theo thứ tự một phần của phương pháp "Symbolic Partial-Order Execution". So với CBMC và Yogar-CBMC, công cụ MVS có lợi thế về phương pháp triển khai và khả năng ứng dụng, cũng như chi phí tính toán thấp hơn. Tuy nhiên, để đảm bảo tính đúng đắn và hiệu quả của chương trình, cần thử nghiệm trên nhiều bài toán hơn và tìm phần cứng đáp ứng yêu cầu để có thể so sánh với hai phương pháp nêu trên.

```
Status: true
Total time: 410.0 (ms)
Z3 result:
unsat
(error line 101 column 10: model is not available)
(:add-rows 84111
:arith-conflicts 1133
:assert-lower 91214
:assert-upper 74492
:binary-propagations 477840
:bound-prop 6436
:conflicts 13017
:decisions 16751
:del-clause 12416
:eliminated-vars 13
:memory 4.49
:minimized-lits 73311
:mk-clause 19543
:pivots 9378
:propagations 928637
:restarts 2
:time 0.39
:total-time 0.41)
```

Hình 4.6: Kết quả kiểm chứng của một bài toán

Chương 5

Kết luận

Khoá luận này đã trình bày một phương pháp và công cụ kiểm chứng chương trình đa luồng bằng thực thi tượng trưng. Phương pháp này đã được chứng minh là một cách tiếp cận hiệu quả để kiểm chứng tính đúng đắn của các chương trình có tính song song cao, đặc biệt là trong việc phát hiện và xử lý các lỗi tiềm ẩn như tranh chấp tài nguyên hay khoá chết. Trước hết, một phương pháp mới dựa trên thực thi tượng trưng đã được đề xuất để kiểm chứng các chương trình đa luồng. Phương pháp này cho phép phát hiện các lỗi tiềm ẩn một cách hiệu quả bằng cách biểu diễn các trạng thái của chương trình dưới dạng công thức logic vị từ và sử dụng bộ giải SMT để xác định tính đúng đắn của các công thức này. Khoá luận cũng đã phát triển công cụ MVS (Multi-threaded Verification Symbolically) dựa trên phương pháp thực thi tượng trưng. Công cụ này đã được thử nghiệm trên các bộ dữ liệu tiêu biểu của cuộc thi SV-COMP và đã cho thấy kết quả khả quan với độ chính xác cao trong việc phát hiện lỗi, với tỉ lệ 16/18 bài toán được kiểm chứng đúng đắn.

Tuy vậy, hiện tại công cụ MVS chỉ có thể xử lý các chương trình không chứa vòng lặp một cách trực tiếp và các lệnh `goto` phức tạp. Điều này giới hạn phạm vi ứng dụng của công cụ trong các chương trình thực tế. Ngoài ra, do hạn chế về phần cứng, khoá luận chưa thể so sánh hiệu suất của MVS với các công cụ kiểm chứng khác như CBMC hay Yogar-CBMC.

Để khắc phục các hạn chế hiện tại, trong tương lai, chúng tôi sẽ tập trung vào việc mở rộng khả năng xử lý của công cụ MVS để hỗ trợ các chương trình chứa các cấu trúc điều khiển phức tạp hơn. Chúng tôi cũng sẽ tìm kiếm các giải pháp phần cứng phù hợp để có thể tiến hành các thử nghiệm so sánh hiệu suất với các công cụ kiểm chứng khác. Cuối cùng, chúng tôi cũng sẽ cố gắng tối ưu hóa hiệu suất của công cụ MVS để giảm thời gian kiểm chứng và tăng khả năng xử lý của công cụ trên các chương trình lớn.

Cuối cùng, khoá luận này không chỉ đóng góp vào lĩnh vực kiểm chứng chương trình đa luồng bằng cách đề xuất một phương pháp mới dựa trên thực thi tượng

trung, mà còn cung cấp một công cụ hữu ích cho các nhà phát triển và kiểm chứng phần mềm. Công cụ MVS có tiềm năng trở thành một giải pháp mạnh mẽ trong việc đảm bảo tính đúng đắn và an toàn của các chương trình đa luồng, đóng góp vào việc nâng cao chất lượng và độ tin cậy của phần mềm.

Tài liệu tham khảo

- [1] James H Fetzter. Program verification: The very idea. *Communications of the ACM*, 31(9):1048–1063, 1988.
- [2] Glenford J Myers, Corey Sandler, and Tom Badgett. *The art of software testing*. John Wiley & Sons, 2011.
- [3] Martin Rinard. Analysis of multithreaded programs. In *International Static Analysis Symposium*, pages 1–19. Springer, 2001.
- [4] Steve Carr, Jean Mayo, and Ching-Kuang Shene. Race conditions: a case study. *Journal of Computing Sciences in Colleges*, 17(1):90–105, 2001.
- [5] Sung-Eun Choi and E Christopher Lewis. A study of common pitfalls in simple multi-threaded programs. In *Proceedings of the thirty-first SIGCSE technical symposium on Computer science education*, pages 325–329, 2000.
- [6] K Rustan M Leino and Peter Müller. A basis for verifying multi-threaded programs. In *European Symposium on Programming*, pages 378–393. Springer, 2009.
- [7] Christoph Von Praun and Thomas R Gross. Static conflict analysis for multi-threaded object-oriented programs. *ACM Sigplan Notices*, 38(5):115–128, 2003.
- [8] Daniel G Waddington, Nilabja Roy, and Douglas C Schmidt. Dynamic analysis and profiling of multithreaded systems. In *Advanced Operating Systems and Kernel Applications: Techniques and Technologies*, pages 156–199. IGI Global, 2010.
- [9] Lucas Cordeiro and Bernd Fischer. Verifying multi-threaded software using smt-based context-bounded model checking. In *Proceedings of the 33rd International Conference on Software Engineering*, pages 331–340, 2011.
- [10] Kari Kähkönen and Keijo Heljanko. Testing multithreaded programs with contextual unfoldings and dynamic symbolic execution. In *2014 14th International Conference on Application of Concurrency to System Design*, pages 142–151. IEEE, 2014.

- [11] Daniel Kroening and Michael Tautschnig. Cbmc-c bounded model checker: (competition contribution). In *Tools and Algorithms for the Construction and Analysis of Systems: 20th International Conference, TACAS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014. Proceedings 20*, pages 389–391. Springer, 2014.
- [12] Liangze Yin, Wei Dong, Wanwei Liu, Yunchou Li, and Ji Wang. Yogar-cbmc: Cbmc with scheduling constraint based abstraction refinement: (competition contribution). In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 422–426. Springer, 2018.
- [13] Konstantin Serebryany and Timur Iskhodzhanov. Threadsanitizer: data race detection in practice. In *Proceedings of the workshop on binary instrumentation and applications*, pages 62–71, 2009.
- [14] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, volume 8, pages 209–224, 2008.
- [15] Miroslav N Velev and Ping Gao. Automatic formal verification of multi-threaded pipelined microprocessors. In *2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 679–686. IEEE, 2011.
- [16] Bil Lewis and Daniel J Berg. *Multithreaded programming with Pthreads*. Prentice-Hall, Inc., 1998.
- [17] Arnab Sinha, Sharad Malik, and Aarti Gupta. Efficient predictive analysis for detecting nondeterminism in multi-threaded programs. In *2012 Formal Methods in Computer-Aided Design (FMCAD)*, pages 6–15. IEEE, 2012.
- [18] Daniel Schemmel, Julian Büning, César Rodríguez, David Laprell, and Klaus Wehrle. Symbolic partial-order execution for testing multi-threaded programs. In *International Conference on Computer Aided Verification*, pages 376–400. Springer, 2020.
- [19] Freark I van der Berg. Llmc: Verifying high-performance software. In *Computer Aided Verification: 33rd International Conference, CAV 2021, Virtual Event, July 20–23, 2021, Proceedings, Part II 33*, pages 690–703. Springer, 2021.

- [20] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Madanlal Musuvathi, Shaz Qadeer, and Thomas Ball. Chess: A systematic testing tool for concurrent software. *Microsoft Research*, 38:39, 2007.
- [21] Maria Christakis, Alkis Gotovos, and Konstantinos Sagonas. Systematic testing for detecting concurrency errors in erlang programs. In *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*, pages 154–163. IEEE, 2013.
- [22] FV Niskov, EA Kutovoy, and Sh F Kurmangaleev. Enhanced s2e for analysis of multi-thread software. *Programming and Computer Software*, 49(Suppl 1):S39–S44, 2023.
- [23] Nicholas Nethercote and Julian Seward. Valgrind: a framework for heavyweight dynamic binary instrumentation. *ACM Sigplan notices*, 42(6):89–100, 2007.
- [24] Ali Jannesari and Walter F Tichy. On-the-fly race detection in multi-threaded programs. In *Proceedings of the 6th workshop on Parallel and distributed systems: testing, analysis, and debugging*, pages 1–10, 2008.
- [25] SV-COMP - international competition on software verification.
- [26] c/pthread · main · SoSy-lab / benchmarking / SV-benchmarks · GitLab.
- [27] c/pthread-nondet · main · SoSy-lab / benchmarking / SV-benchmarks · GitLab.
- [28] c/pthread-deagle · main · SoSy-lab / benchmarking / SV-benchmarks · GitLab.
- [29] c/pthread-atomic · main · SoSy-lab / benchmarking / SV-benchmarks · GitLab.
- [30] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008.